

Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

August 2, 2026

Contents

1	Thesis Plan	1
1.1	Plan Inspiration	1
1.2	Working Title	1
1.3	Central Thesis Question	1
1.4	Research Questions	1
1.5	Expected Contributions	1
1.6	Chapter Outline	2
1.7	Appendix Plan	4
1.8	Immediate Writing Order	4
1.9	Open Decisions	4
2	Methods	5
2.1	Scaling and Normalization	5
2.1.1	Deterministic physical scaling	5
2.1.2	Running standardization	6
2.2	Graph Representations	6
2.2.1	Busbar Representation	6
2.2.2	Disaggregated Representation	8
2.2.3	Disaggregated Representation with Line Nodes	10
2.2.4	Local actor graphs and the global state graph	14
2.2.5	Graph encoder	14
2.3	Substation Representation: Direct Edges and Summary Nodes	14
2.3.1	Direct same-substation edges (e)	15
2.3.2	Substation summary nodes (n)	15
2.4	Pooling and Graph Readout	16
2.4.1	Mean, Max, Sum pooling	16
2.4.2	Virtual-node graph readout	16
2.5	Candidate-Action Scoring from the Explicit-Line Graph	18
2.5.1	Action-index head versus candidate scoring	18
2.5.2	Candidate-action metadata	18
2.5.3	Node embeddings from the encoder	19
2.5.4	Typed candidate pooling	19
2.5.5	Shared scorer and idle action	20
2.5.6	Configuration, cost, and current limits	20
2.6	Graph Actor and Critic Configuration	21
3	Graph-Design Screening	22
3.1	How This Chapter Is Organized	22
3.2	Shared Protocol	22
3.2.1	Reference configuration	23
3.2.2	Endpoint statistics	23
3.2.3	Compute	23
3.3	Screen A: Encoder Depth and Width	23
3.4	Screen B: Encoder Operator and Readout	25
3.5	Screen C: Structural Augmentations of the Busbar Graph	27
3.6	Screen D: Generator and Load Message Direction	30
3.7	Campaign Status	32

4	Experimental Results	34
4.1	How to Read This Chapter	34
4.2	Evaluation Protocol	34
4.3	Experiment Index	34
4.4	IEEE 14-Bus Experiments	35
4.4.1	A0 Core Reruns	35
4.4.2	Initial Do-Nothing Bias	37
4.4.3	GINE Encoder Reruns	38
4.4.4	Heavy GINE	41
4.5	WCCI Action-Space Reduction	45
4.6	Decentralized Sparse Intervention Control	45
4.6.1	Evaluation-Time Rho Heuristics	46
4.6.2	Intervention-Gated Actor	46
4.6.3	Fixed Sparse-Action Penalty	47
4.6.4	Adaptive Intervention Budget	47
4.7	Current Takeaway	49
A	GINE and GAT Graph-Actor Architectures	51
A.1	Shared input and encoder pipeline	51
A.2	GINE message passing	51
A.3	GAT message passing	52
A.4	Pooling and graph output	52
A.5	Agent-specific action head	53
A.6	Exact node inputs by representation	53
A.7	Exact edge inputs by representation	53
B	Full-Test Checkpoint Registry	55

Chapter 1

Thesis Plan

1.1 Plan Inspiration

This plan follows the structure of the reference thesis in `mAI2024deMo1B.pdf`: start from the power-grid motivation and research questions, introduce the scientific background, describe the environment before the algorithms, then separate methods from results and close by answering the research questions. The adapted structure below is tuned to this repository: Grid2Op topology control, decentralized MAPPO, graph encoders, rerun ablations, and sparse intervention mechanisms.

1.2 Working Title

Decentralized Multi-Agent Reinforcement Learning for Sparse Power Grid Topology Control

1.3 Central Thesis Question

How can decentralized multi-agent reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces and avoiding unnecessary interventions?

1.4 Research Questions

1. How should the Grid2Op topology-control problem be formulated as a decentralized multi-agent decision problem?
2. Which architectural choices improve survival and generalization: flat MLP policies, graph neural network encoders, shared actor weights, critic design, or optimized critic updates?
3. How do initialization and training choices, especially the initial do-nothing bias and reward normalization, affect exploration and learned behavior?
4. Can sparse-control mechanisms, such as rho-based overrides, intervention gates, fixed penalties, or adaptive intervention budgets, reduce unnecessary topology actions without sacrificing episodic survival?

1.5 Expected Contributions

- A clear decentralized formulation of topology optimization in Grid2Op.
- A controlled comparison of MAPPO baselines, GNN encoders, critic variants, and initialization choices.
- An ablation study of sparse intervention mechanisms for more operator-like control.
- A reproducible experiment and analysis pipeline based on W&B histories, cached metrics, notebooks, and final evaluation scripts.

1.6 Chapter Outline

Chapter 1: Introduction

Goal. Motivate topology control as a low-cost but difficult operational tool for power-grid reliability.

Planned content.

- Energy-transition context: higher demand, renewable uncertainty, and stressed transmission networks.
- Topology control as an alternative or complement to redispatching and grid expansion.
- Why the problem is hard: sequential control, physical constraints, rare failures, and combinatorial topology actions.
- Why decentralized MARL is a natural fit: local substations, regional decision making, reduced per-agent action spaces, and scalability.
- Research questions, contributions, and thesis outline.

Chapter 2: Background and Related Work

Goal. Give the reader enough power-systems and RL background to understand the design choices.

Planned content.

- Power-grid operation: substations, busbars, line loading, overloads, contingencies, and remedial actions.
- Grid2Op and the Learning to Run a Power Network setting.
- Reinforcement learning basics: MDPs, policy gradients, PPO, and MAPPO.
- Multi-agent reinforcement learning: decentralized execution, centralized training, credit assignment, and non-stationarity.
- Graph neural networks for grid-structured observations.
- Safe, sparse, and constrained RL ideas relevant to intervention budgets.

Chapter 3: Environment and Problem Formulation

Goal. Define exactly what the agents observe, choose, optimize, and are evaluated on.

Planned content.

- Grid2Op environment used in the repository and the selected chronics.
- Multi-agent decomposition: agent-region or agent-substation assignment.
- Observation spaces: local grid state, line loadings, topology state, and optional graph observations.
- Action spaces: local topology actions, action 0 as do nothing, legal action constraints, and action-space reduction.
- Reward and evaluation metrics: episodic survival, illegal action rate, action-0 fraction, non-idle fraction, and train/test chronic split.
- Reproducibility setup: seeds, W&B logging, cached histories, and deterministic evaluation.

Chapter 4: Methods

Goal. Present the implemented learning systems and the ablations.

Planned content.

- MAPPO baseline: actor, critic, rollout collection, PPO objective, and centralized training assumptions.
- Baseline MLP agents and the A0 family of reruns.
- Graph-encoder agents: bus graph construction, GINE/GAT/GCN variants, node and edge pre-encoders, graph readout, and flat-feature concatenation.

- Action heads: the action-index MLP head and the candidate-action head that scores each action from the explicit-line graph nodes it touches.
- Critic variants: GNN critic versus MLP critic, legacy versus optimized critic updates.
- Initialization and exploration choices: entropy coefficient, entropy decay, and initial do-nothing probability.
- Sparse-control mechanisms: evaluation-time rho heuristic, intervention gate, fixed sparse-action penalties, and adaptive intervention budgets.
- Experimental protocol for ablations: controlled seeds, fixed evaluation cadence, and fair baseline comparisons.

Chapter 5: Experiments and Results

Goal. Show which design choices matter and what behaviors the agents learn.

Planned content.

- Main baseline performance: training stability and test survival.
- GNN architecture ablations: concat-flat, actor sharing, GINE versus GAT or weighted GCN, light versus full encoders, and critic design.
- Action-head ablation: candidate-action scoring versus the action-index head on the explicit-line graph, on bus14 first and then on the reduced WCCI action space.
- A0 rerun ablations: optimized critic updates, smaller MLP, reward normalization, and do-nothing initialization bias.
- Sparse-control ablations: fixed penalty, global-safe weighting, local-safe adaptive budget, rho-threshold sensitivity, and budget target sensitivity.
- Behavioral analysis: how often agents act, whether interventions are local, and whether high survival comes with excessive topology churn.
- Generalization: seen versus unseen chronics and robustness across seeds.

Figures and tables to prepare.

- Table of run families and what each ablation changes.
- Survival curves for baseline, GNN, A0, and sparse-control comparisons.
- Action-frequency plots: action 0 fraction, any non-idle action, and multi-agent simultaneous interventions.
- Summary table with final survival, peak survival, and action sparsity.

Chapter 6: Discussion

Goal. Interpret the results rather than just restating them.

Planned content.

- Answer each research question directly.
- Explain which components appear essential, useful, or unnecessary.
- Discuss the tradeoff between survival and sparse interventions.
- Relate learned behavior to operator-like control: act rarely, act locally, and act when the grid is unsafe.
- Discuss limits: simulator scope, incomplete operational constraints, seed variance, training cost, and dependency on chosen chronics.

Chapter 7: Conclusion and Future Work

Goal. Close with the main thesis claim and next research directions.

Planned content.

- Short summary of the problem, approach, and core findings.

- Final conclusion about decentralized MARL for topology control.
- Future work: richer graph representations, larger grids, outage contingencies, constrained RL tuning, better coordination mechanisms, and operational validation.

1.7 Appendix Plan

- Appendix A: Environment details and action-space construction.
- Appendix B: Hyperparameters and training configuration tables.
- Appendix C: Full ablation run registry.
- Appendix D: Additional survival and behavior plots.
- Appendix E: Reproducibility notes for notebooks, cached W&B histories, and evaluation scripts.

1.8 Immediate Writing Order

1. Write Chapter 3 first, because it fixes notation and the experiment vocabulary.
2. Write Chapter 4 next, using the existing configuration READMEs as source notes for method variants.
3. Draft the Chapter 5 table of ablations before prose, so the results chapter has a stable spine.
4. Write Chapter 1 once the main conclusion is clearer.
5. Finish Chapters 6 and 7 after the final plots and summary metrics are frozen.

1.9 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Methods

This chapter describes the graph inputs and graph-actor variants used in the experiments. The presentation separates four design choices: feature scaling, the base graph representation, substation-level augmentations, and graph pooling.

2.1 Scaling and Normalization

Two independent transformations can be applied to the continuous node and edge features of every graph schema: deterministic physical scaling followed by optional running standardization. Let $x_{i,f,t}$ be feature f of node or edge i at time t . The complete pipeline is

$$x_{i,f,t}^{\text{scale}} = \begin{cases} x_{i,f,t}/s_f, & \text{if physical scaling is enabled,} \\ x_{i,f,t}, & \text{otherwise,} \end{cases} \quad (2.1)$$

$$\hat{x}_{i,f,t} = \begin{cases} \text{clip}\left(\frac{x_{i,f,t}^{\text{scale}} - \mu_{f,t}}{\sqrt{\sigma_{f,t}^2 + \epsilon}}, -c, c\right), & \text{if running normalization is enabled,} \\ x_{i,f,t}^{\text{scale}}, & \text{otherwise.} \end{cases} \quad (2.2)$$

Here s_f is a fixed physical reference and $\mu_{f,t}$ and $\sigma_{f,t}^2$ are running moments estimated during training. When both transformations are enabled, the moments describe x/s_f . Clipping applies only to running standardization; $c = 10$ by default and a non-positive value disables clipping.

2.1.1 Deterministic physical scaling

Physical scaling divides each quantity by a constant with the same unit and therefore preserves physical zero. Table 2.1 gives the exact constants. The power base may be specified explicitly; otherwise the largest installed generator rating is used, with a 100 MW fallback when the environment does not expose generator ratings.

Table 2.1: Deterministic physical scales used for continuous graph features.

Features	Scale s_f	Source
<code>gen_p</code> , <code>load_p</code> , <code>p</code>	Power base in MW	Configured value or maximum generator rating.
<code>gen_theta</code> , <code>load_theta</code> , <code>theta</code>	180 degrees	Fixed angular scale.
<code>time_before_cooldown_sub</code>	Maximum substation cooldown	Grid2Op environment parameter.
<code>time_before_cooldown_line</code>	Maximum line cooldown	Grid2Op environment parameter.
<code>timestep_overflow</code>	Allowed overflow duration	Grid2Op environment parameter.
Maintenance time and duration	Episode horizon	Grid2Op chronic horizon.

2.1.2 Running standardization

Running moments are estimated with Welford’s online algorithm. One normalizer is shared across agents and updated from the full state graph once per environment step. Heterogeneous graphs maintain separate node statistics for each node type. Edge statistics include only active physical-line relations; inactive candidates and structural, asset-attachment, same-substation, and virtual relations do not contribute.

The loading ratio `rho` is already dimensionless and is neither rescaled nor standardized. Preserving it is also necessary when GCN uses `rho` directly as a non-negative message weight. Binary connection states, node and edge masks, domain masks, type indicators, and relation one-hot attributes are also left unchanged. Only the continuous channels listed in Table 2.1 enter the running statistics. The running moments are frozen during evaluation.

2.2 Graph Representations

The graph observation can use one of three base representations. The `bus` representation aggregates equipment values onto busbar nodes. The `heterogeneous` representation disaggregates generators and loads into explicit asset nodes while keeping physical lines as busbar-to-busbar edges. The `heterogeneous_line` representation further disaggregates transmission lines into explicit line nodes.

Grid2Op exposes the instantaneous electrical state and topology through a structured observation [1]. For each representation, the graph construction keeps the set of candidate nodes and edges fixed throughout an episode while using a dynamic edge mask to express the topology that is electrically active at time step t .

For agent a , the graph observation can be written as

$$\mathcal{G}_t^{(a)} = \left(\mathcal{V}^{(a)}, \mathcal{E}_{\text{cand}}^{(a)}, \mathbf{X}_t^{(a)}, \mathbf{Z}_t^{(a)}, \mathbf{m}_t^{(a)} \right), \quad (2.3)$$

where $\mathcal{V}^{(a)}$ and $\mathcal{E}_{\text{cand}}^{(a)}$ are static, $\mathbf{X}_t^{(a)}$ contains node features, $\mathbf{Z}_t^{(a)}$ contains edge features, and $\mathbf{m}_t^{(a)}$ indicates which candidate edges are active. This separates the physical equipment layout, which does not change during an episode, from the busbar assignment and line status, which can change after every topology action.

The following subsections define the three schemas.

2.2.1 Busbar Representation

The `bus` schema is the compact busbar representation. Busbars are the only nodes, active transmission lines form the graph backbone, and generator and load quantities are aggregated into their currently assigned busbars.

Busbar nodes

Let S denote the number of substations and let B denote the maximum number of busbars per substation. A node is created for every substation–busbar pair, including busbars to which no asset is connected at the current time. A global node identifier is assigned as

$$i(s, b) = sB + b, \quad s \in \{0, \dots, S-1\}, \quad b \in \{0, \dots, B-1\}. \quad (2.4)$$

The complete graph therefore contains SB nodes. Grid2Op uses one-based busbar identifiers in its topology vector; these identifiers are converted to the zero-based index b in Equation 2.4.

Table 2.2 lists the six node features. Generator and load quantities are assigned to the busbar specified by the current topology vector. Active powers are summed when several assets occupy the same busbar, whereas the corresponding angles are averaged. Disconnected assets do not contribute to any node. The substation cooldown is repeated on every busbar node of that substation.

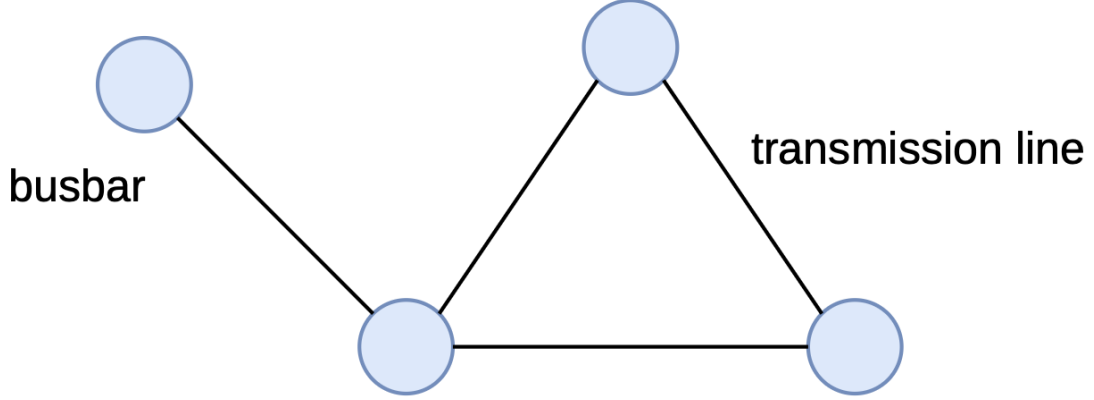


Figure 2.1: Busbar graph representation. Blue nodes denote busbars and black connections denote active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

Table 2.2: Features attached to each busbar node.

Feature	Definition
<code>gen_p</code>	Sum of active generator power connected to the busbar.
<code>gen_theta</code>	Mean voltage angle of generators connected to the busbar.
<code>load_p</code>	Sum of active load power connected to the busbar.
<code>load_theta</code>	Mean voltage angle of loads connected to the busbar.
<code>time_before_cooldown_sub</code>	Remaining substation cooldown.
<code>domain_mask</code>	One for a substation controlled by the agent and zero for a contextual neighbor.

This aggregation produces a compact representation but deliberately removes the identities and counts of individual generators and loads.

Candidate line edges and dynamic topology

Consider a physical line ℓ whose origin and extremity belong to substations $o(\ell)$ and $e(\ell)$. For every pair of possible endpoint busbars (b_o, b_e) , the graph reserves the two directed candidate edges

$$i(o(\ell), b_o) \rightarrow i(e(\ell), b_e) \quad \text{and} \quad i(e(\ell), b_e) \rightarrow i(o(\ell), b_o). \quad (2.5)$$

Thus, each physical line contributes $2B^2$ candidate directed edges. The two directions share the same line attributes. For the usual case $B = 2$, one line produces eight candidate edges even though at most two—one per direction—are electrically active at a given time.

The mask for a candidate endpoint assignment is

$$m_{\ell, b_o, b_e, t} = \mathbf{1}[\text{line } \ell \text{ is connected at } t] \mathbf{1}[b_{\ell, t}^{\text{or}} = b_o] \mathbf{1}[b_{\ell, t}^{\text{ex}} = b_e]. \quad (2.6)$$

If a line is disconnected, all of its candidate edges are masked. Otherwise, only the pair corresponding to its current origin and extremity busbars remains active. Inactive edge features are set to zero, and inactive edges are removed before message passing. Topology changes can therefore modify graph connectivity without rebuilding the candidate graph.

The dynamic attributes copied onto active physical-line edges are given in Table 2.3. The two maintenance features are present only for environments with scheduled maintenance.

Table 2.3: Features attached to physical-line edges.

Feature	Definition
<code>line_status</code>	Binary connection status of the physical line.
<code>rho</code>	Line loading relative to its thermal limit.
<code>timestep_overflow</code>	Number of consecutive time steps in overload.
<code>time_before_cooldown_line</code>	Remaining cooldown before another line action is legal.
<code>time_next_maintenance</code>	Time until scheduled maintenance, when available.
<code>duration_next_maintenance</code>	Duration of scheduled maintenance, when available.

For GINE and GAT, the edge embedding of a busbar-to-busbar transmission edge is therefore exactly the row of Table 2.3. Optional self and same-substation edges use the same feature width: all physical channels are zero except `rho`= 1, which is a neutral structural value. The homogeneous schema does not append relation one-hots, so these optional relations are distinguished only by their endpoints and neutral physical attributes. **Inactive candidate edges are removed before either encoder is called; their zero feature rows never generate messages.**

2.2.2 Disaggregated Representation

The **heterogeneous** schema keeps busbars as the backbone and adds one node per generator and load. Transmission lines remain busbar-to-busbar edges. The node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}}, \quad (2.7)$$

with typed directed relations between these sets. Generator and load states are therefore preserved individually.

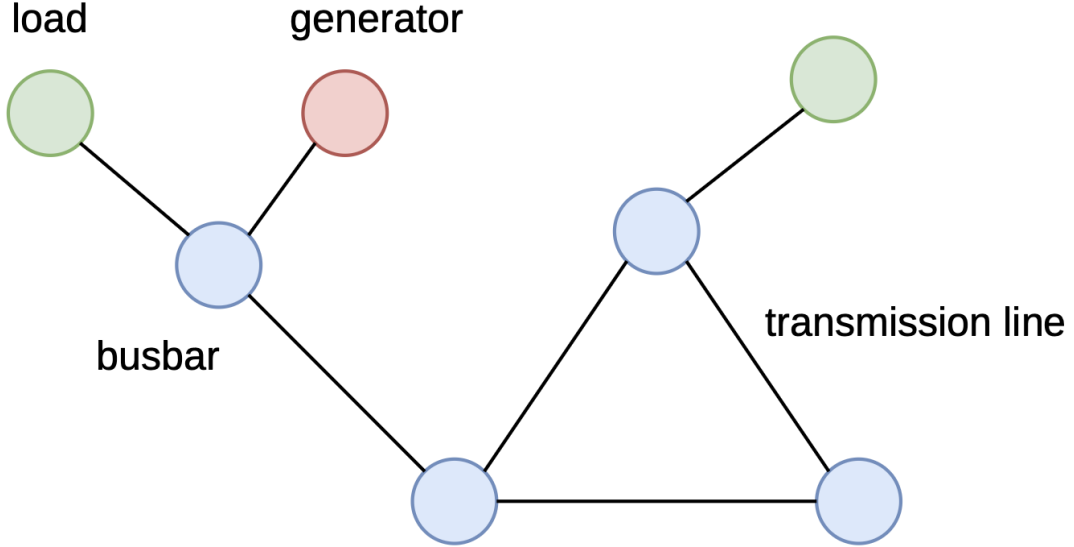


Figure 2.2: Heterogeneous equipment representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

End-to-end architecture

With bidirectional asset and line relations, information follows the path

$$\text{generator} \leftrightarrow \text{local busbar} \leftrightarrow \text{remote busbar} \leftrightarrow \text{load}. \quad (2.8)$$

One message-passing layer crosses one arrow. Generator or load information therefore reaches its local busbar after one layer and the adjacent remote busbar after two. With one-way asset relations, the reverse flow is absent. **Mean readout includes all nodes in the local graph, including one-hop context.**

Typed nodes and features

For a graph with S included substations, B busbars per substation, N_g included generators, and N_d included loads, the number of nodes is

$$|\mathcal{V}| = SB + N_g + N_d. \quad (2.9)$$

Busbar nodes are created exactly as in Section 2.2.1. In addition, each generator and load whose substation is present in the graph receives its own node. **Asset nodes remain present when an asset is disconnected, so the node set stays fixed throughout an episode.**

Every busbar, generator, and load node is represented by the same ordered eight-dimensional feature vector

$$\mathbf{x}_v = [\text{is_busbar}, \text{is_generator}, \text{is_load}, \text{p}, \text{theta}, \text{time_before_cooldown_sub}, \text{connected}, \text{domain_mask}]^T \in \mathbb{R}^8. \quad (2.10)$$

The width, column order, and meaning of this vector are identical for all node types; node type does not select a smaller feature vector. Table 2.4 shows the value placed in every shared column. A zero in the table means that the column is still present for that node type but is explicitly zero-filled.

Table 2.4: Values placed in the eight feature columns shared by every node in the direct-line heterogeneous graph. Disconnected assets retain their node row but have zero power and angle.

Shared feature column	Busbar node	Generator node	Load node
is_busbar	1	0	0
is_generator	0	1	0
is_load	0	0	1
p	0	gen_p if connected; 0 otherwise	load_p if connected; 0 otherwise
theta	0	gen_theta if connected; 0 otherwise	load_theta if connected; 0 otherwise
time_before_cooldown_sub	Substation value	0	0
connected	1	1 connected; 0 disconnected	1 connected; 0 disconnected
domain_mask	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual

Typed relations

Every candidate directed edge is represented by the same ordered feature vector. Its physical block is

$$\mathbf{p}_{uv} = [\text{line_status}, \text{rho}, \text{timestep_overflow}, \text{time_before_cooldown_line}, (\text{time_next_maintenance}, \text{duration_next_maintenance})_{\text{optional}}]^T. \quad (2.11)$$

The two maintenance columns are appended only when scheduled maintenance is available. The relation block is the same seven-column vector for every edge:

$$\mathbf{r}_{uv} = [\text{relation_self}, \text{relation_physical_line}, \text{relation_same_substation_busbar}, \text{relation_generator_to_busbar}, \text{relation_busbar_to_generator}, \text{relation_load_to_busbar}, \text{relation_busbar_to_load}]^T. \quad (2.12)$$

The complete common edge vector is therefore

$$\mathbf{e}_{uv} = [\mathbf{p}_{uv}^T \mid \mathbf{r}_{uv}^T]^T \in \begin{cases} \mathbb{R}^{11}, & \text{without maintenance columns,} \\ \mathbb{R}^{13}, & \text{with maintenance columns.} \end{cases} \quad (2.13)$$

Its width, column order, and meaning do not change with the relation type. An active physical-line edge fills $\mathbf{p}_{uv}$ with its measured line state. Every active structural or asset-attachment edge instead uses

$$\mathbf{p}_{\text{neutral}} = [0, 1, 0, 0, (0, 0)_{\text{optional}}]^T, \quad (2.14)$$

so only the shared **rho** column is nonzero. In every active edge row, exactly one column of $\mathbf{r}_{uv}$ is one and the other six are zero.

Table 2.5 shows the values placed in these common blocks for every relation. An inactive candidate retains the same vector width, but its entire row is zero-filled and **edge_mask**= 0 removes it before GINE or GAT message passing.

Table 2.5: Values placed in the common edge-feature vector shared by every active directed relation in the direct-line heterogeneous graph. “Other six” refers to the remaining columns of the seven-column relation block.

Directed relation	Active rule	Shared physical block	Shared relation block
$v \rightarrow v$ (self)	Optional; always active when enabled	$\mathbf{p}_{\text{neutral}}$	relation_self = 1; other six = 0
$i_o \rightarrow i_e$ and reverse (physical line)	Line online and candidate busbars match the current endpoints	Measured $\mathbf{p}_{uv}$	relation_physical_line = 1; other six = 0
$i(s, b) \rightarrow i(s, b'), b \neq b'$	Optional; every ordered same-substation busbar pair	$\mathbf{p}_{\text{neutral}}$	relation_same_substation_busbar = 1; other six = 0
$g \rightarrow i(s(g), b)$	Generator connected to candidate busbar b ; direction enabled	$\mathbf{p}_{\text{neutral}}$	relation_generator_to_busbar = 1; other six = 0
$i(s(g), b) \rightarrow g$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	relation_busbar_to_generator = 1; other six = 0
$d \rightarrow i(s(d), b)$	Load connected to candidate busbar b ; direction enabled	$\mathbf{p}_{\text{neutral}}$	relation_load_to_busbar = 1; other six = 0
$i(s(d), b) \rightarrow d$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	relation_busbar_to_load = 1; other six = 0

Generator and load directions are selected independently as **bidirectional**, **asset_to_busbar**, or **busbar_to_asset**. The default is **bidirectional**. Physical-line and same-substation relations remain **bidirectional**.

Physical lines use Equation 2.5. For generator g at substation $s(g)$, the bidirectional candidates are

$$g \rightarrow i(s(g), b) \quad \text{and} \quad i(s(g), b) \rightarrow g, \quad b \in \{0, \dots, B-1\}. \quad (2.15)$$

One-way modes retain only one member of each pair. Loads use the same rule. A disconnected asset has no active attachment.

Let $d_g, d_d \in \{1, 2\}$ denote the selected number of directions for generator and load attachments. Without optional self and same-substation relations, the candidate edge count is

$$|\mathcal{E}_{\text{cand}}| = 2B^2L + B(d_g N_g + d_d N_d), \quad (2.16)$$

where L is the number of included physical lines. Enabling both optional relations adds $|\mathcal{V}|$ self-edges and $SB(B-1)$ directed same-substation busbar edges.

Local disaggregated graphs

A local actor graph contains controlled substations plus optional one-hop neighbors. Nodes belonging to the controlled substations form the action domain, while the additional neighbors provide context. The state graph contains the full grid. Relative to the busbar graph, this schema adds $N_g + N_d$ nodes and between $B(N_g + N_d)$ and $2B(N_g + N_d)$ asset candidates, but preserves individual asset states.

2.2.3 Disaggregated Representation with Line Nodes

The **heterogeneous_line** schema replaces each direct physical-line edge with a transmission-line node between its endpoint busbars. Its node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}} \cup \mathcal{V}_{\text{line}}. \quad (2.17)$$

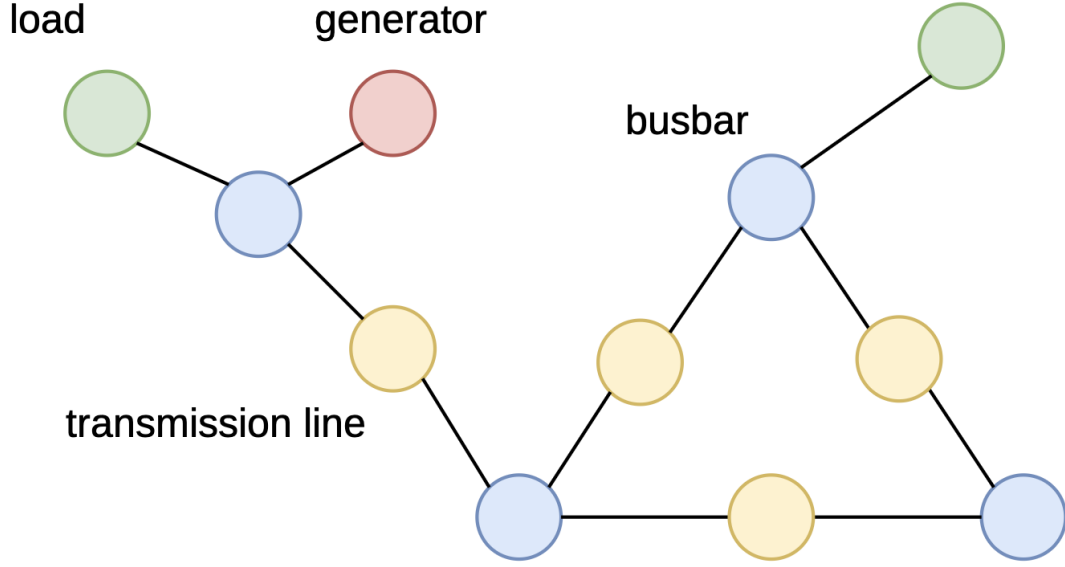


Figure 2.3: Heterogeneous representation with explicit transmission-line nodes. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain explicit terminal nodes.

Shared node features

For S included substations, B busbars per substation, N_g generators, N_d loads, and L included transmission lines, the graph contains

$$|\mathcal{V}| = SB + N_g + N_d + L \quad (2.18)$$

nodes. A generator or load node remains allocated when it is disconnected, and a transmission-line node remains allocated when the line is offline.

Every node type uses the same ordered feature vector:

$$\begin{aligned} \mathbf{x}_v = & [\text{is_busbar}, \text{is_generator}, \text{is_load}, \text{is_transmission_line}, \\ & \text{p}, \text{theta}, \text{line_status}, \text{rho}, \text{timestep_overflow}, \\ & \text{time_before_cooldown_line}, \\ & (\text{time_next_maintenance}, \text{duration_next_maintenance})_{\text{optional}}, \\ & \text{time_before_cooldown_sub}, \text{connected}, \text{domain_mask}]^T, \\ \mathbf{x}_v \in & \begin{cases} \mathbb{R}^{13}, & \text{without maintenance columns,} \\ \mathbb{R}^{15}, & \text{with maintenance columns.} \end{cases} \end{aligned} \quad (2.19)$$

The width and column order are identical for busbar, generator, load, and transmission-line nodes. Table 2.6 specifies every entry; 0 denotes an explicitly zero-filled column, not an omitted feature. For generators and loads, power and angle are set to zero when the asset is disconnected.

Table 2.6: Values placed in the common node-feature vector of the explicit-line heterogeneous graph. The two maintenance rows are included only when maintenance observations are enabled.

Shared feature column	Busbar	Generator	Load	Transmission line
<code>is_busbar</code>	1	0	0	0
<code>is_generator</code>	0	1	0	0
<code>is_load</code>	0	0	1	0
<code>is_transmission_line</code>	0	0	0	1
<code>p</code>	0	<code>gen_p</code> if connected; 0 otherwise	<code>load_p</code> if connected; 0 otherwise	0
<code>theta</code>	0	<code>gen_theta</code> if connected; 0 otherwise	<code>load_theta</code> if connected; 0 otherwise	0
<code>line_status</code>	0	0	0	Observed line value
<code>rho</code>	0	0	0	Observed ρ_ℓ
<code>timestep_overflow</code>	0	0	0	Observed line value
<code>time_before_cooldown_line</code>	0	0	0	Observed line value
<code>time_next_maintenance</code>	0	0	0	Observed line value
<code>duration_next_maintenance</code>	0	0	0	Observed line value
<code>time_before_cooldown_sub</code>	Substation value	0	0	0
<code>connected</code>	1	1 connected; 0 disconnected	1 connected; 0 disconnected	1 online; 0 offline
<code>domain_mask</code>	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual	1 if either endpoint is controlled; 0 otherwise

A disconnected line therefore keeps its node row. Its `line_status` and `connected` entries are zero, its busbar attachments are masked, and the remaining observed line-state entries, including cooldown and maintenance, remain in the line-node vector.

Shared edge features and endpoint relations

Let $o(\ell)$ and $e(\ell)$ denote the origin and extremity substations of line ℓ . For every busbar index b , the fixed candidate graph contains

$$i(o(\ell), b) \leftrightarrow \ell \quad \text{and} \quad i(e(\ell), b) \leftrightarrow \ell, \quad b \in \{0, \dots, B-1\}. \quad (2.20)$$

The line direction is `bidirectional`, `line_to_busbar`, or `busbar_to_line`. An online line has four active attachments in the bidirectional case and two in a one-way case. The mask selects the current origin and extremity busbars.

Every candidate directed edge uses the same ordered ten-dimensional feature vector:

$$\mathbf{e}_{uv} = [\text{relation_self}, \text{relation_same_substation_busbar}, \\ \text{relation_busbar_to_line_origin}, \text{relation_line_origin_to_busbar}, \\ \text{relation_busbar_to_line_extremity}, \text{relation_line_extremity_to_busbar}, \\ \text{relation_generator_to_busbar}, \text{relation_busbar_to_generator}, \\ \text{relation_load_to_busbar}, \text{relation_busbar_to_load}]^T \in \mathbb{R}^{10}. \quad (2.21)$$

These edges contain no continuous line-state attributes because the transmission-line node already contains `rho`, `status`, `overflow`, `cooldown`, and `maintenance`. For an active edge, exactly one relation column is one and the other nine are zero. An inactive candidate has all ten entries set to zero and is removed by `edge_mask`= 0 before message passing. Table 2.7 gives the active rule and the nonzero entries for every relation.

Table 2.7: Nonzero entries in the common 10-column edge-feature vector of the explicit-line heterogeneous graph. All relation columns not named in a row are zero.

Directed relation	Active rule	Relation column set to 1
$v \rightarrow v$ (self)	Optional; always active when enabled	<code>relation_self</code>
$i(s, b) \rightarrow i(s, b'), b \neq b'$	Optional; every ordered same-substation pair	<code>relation_same_substation_busbar</code>
$i_o \rightarrow \ell$	Online line, current origin busbar, direction enabled	<code>relation_busbar_to_line_origin</code>
$\ell \rightarrow i_o$	Same mask; reverse direction enabled	<code>relation_line_origin_to_busbar</code>
$i_e \rightarrow \ell$	Online line, current extremity busbar, direction enabled	<code>relation_busbar_to_line_extremity</code>
$\ell \rightarrow i_e$	Same mask; reverse direction enabled	<code>relation_line_extremity_to_busbar</code>
$g \rightarrow i(s(g), b)$	Generator attached to b ; direction enabled	<code>relation_generator_to_busbar</code>
$i(s(g), b) \rightarrow g$	Same mask; reverse direction enabled	<code>relation_busbar_to_generator</code>
$d \rightarrow i(s(d), b)$	Load attached to b ; direction enabled	<code>relation_load_to_busbar</code>
$i(s(d), b) \rightarrow d$	Same mask; reverse direction enabled	<code>relation_busbar_to_load</code>

Origin, extremity, and direction have separate relation types. Generator and load attachments follow Equation 2.15; self and same-substation relations remain optional.

Message-passing path through a line node

The direct-line graph connects endpoint busbars in one hop. The explicit-line graph uses

$$i_o \rightarrow \ell \rightarrow i_e, \quad (2.22)$$

so endpoint exchange requires two layers. The first layer forms

$$\mathbf{h}_\ell^{(1)} = f_\ell(\mathbf{h}_\ell^{(0)}, \mathbf{h}_{i_o}^{(0)}, \mathbf{h}_{i_e}^{(0)}), \quad (2.23)$$

and the second returns this joint state to the endpoints. Later layers enlarge the receptive field. The edges around the line node are bidirectional: the line gathers both endpoints and sends the updated state back; endpoint exchange is possible;

Let $d_\ell \in \{1, 2\}$ denote the number of selected line-attachment directions and retain d_g, d_d from the preceding section. Without the optional relations, the number of candidate directed edges is

$$|\mathcal{E}_{\text{cand}}| = 2Bd_\ell L + B(d_g N_g + d_d N_d). \quad (2.24)$$

For the common case $B = 2$, the line-attachment term has the same size as the $2B^2L$ direct busbar-to-busbar candidate set in Equation 2.16. For $B > 2$, explicit line nodes make this term grow linearly rather than quadratically with the number of busbars.

Local disaggregated-line graphs

Local graphs use the same controlled-domain and one-hop rules. A line is marked controlled when either endpoint is controlled. The state graph contains the full grid. Relative to **heterogeneous**, this schema adds L nodes and one message-passing hop between adjacent busbars.

Architectural comparison of the two disaggregated schemas

Table 2.8 summarizes the structural differences between the two heterogeneous schemas.

Table 2.8: Architectural comparison of the disaggregated graph schemas.

Aspect	heterogeneous	heterogeneous_line
Node types	Busbar, generator, load	Busbar, generator, load, transmission line
Physical line	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Features of the line node
Endpoint path	One hop between busbars	Two hops through the line node
Minimum useful depth	One layer for adjacent busbar exchange; two for asset-to-remote-busbar flow	Two layers for adjacent busbar exchange through a line node
Hidden line state	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Graph size	Fewer nodes	Adds L line nodes

2.2.4 Local actor graphs and the global state graph

Each agent controls a set of substations $\mathcal{C}_a$. Under decentralized observability, its local graph can be constructed in two ways. Without neighbor context, the node set contains only substations in $\mathcal{C}_a$, and the line set contains only lines whose two endpoints lie inside this region. With one-hop context enabled, the graph additionally contains every line touching $\mathcal{C}_a$ and the substations at the opposite endpoints of those lines. The `domain_mask` node feature and a separate controlled-node mask distinguish the controlled region from contextual neighbors. **This second mask also permits graph readout to pool only the nodes on which the agent can act.**

The full-grid state graph is available to the centralized critic during training, while each actor receives only its local graph. This follows the centralized-training, decentralized-execution structure of MAPPO [2].

2.2.5 Graph encoder

GINE and GAT receive every column of the edge-feature vector. The homogeneous graph provides only physical-line columns; the heterogeneous graphs append the relation one-hot columns defined in Tables 2.5 and 2.7. Node types are encoded by the explicit `is_*` node features.

GINE and GAT use the same high-level actor pipeline:

$$\begin{aligned}
 (\mathbf{X}, \mathbf{E}, \mathbf{m}) &\longrightarrow \text{active-edge filtering} \longrightarrow \text{node projection} \longrightarrow \mathbf{H}^{(0)}, \\
 \mathbf{H}^{(0)} &\longrightarrow K \text{ message-passing layers} \longrightarrow \text{graph readout} \longrightarrow \text{action logits}.
 \end{aligned}
 \tag{2.25}$$

They differ only in the message-passing block. GINE inserts the projected edge vector into the content of every message. GAT uses the projected edge vector to compute the attention weight assigned to that message. Pooling, the graph output projection, and the action head are otherwise identical. The exact equations and architectural dimensions are given in Appendix A.

The busbar schema aggregates generator and load states and omits reactive power, voltage magnitudes, and directional endpoint flows. Empty busbars remain in the node set. The preprocessing in Section 2.1 changes feature values but not the graph schema.

2.3 Substation Representation: Direct Edges and Summary Nodes

Two optional factors change how information is exchanged inside a substation. The factor e adds direct same-substation busbar edges. The factor n adds one learned substation-summary node per included substation.

2.3.1 Direct same-substation edges (e)

In addition to physical-line relations, the graph may include a self-edge $v \rightarrow v$ for every node and a directed same-substation edge $i(s, b) \rightarrow i(s, b')$ for every $b \neq b'$. These relations remain active throughout the episode and use zero physical-line features except for the neutral value $\mathbf{rho} = 1$. Same-substation edges permit message passing between alternative busbars but do not represent an electrical connection.

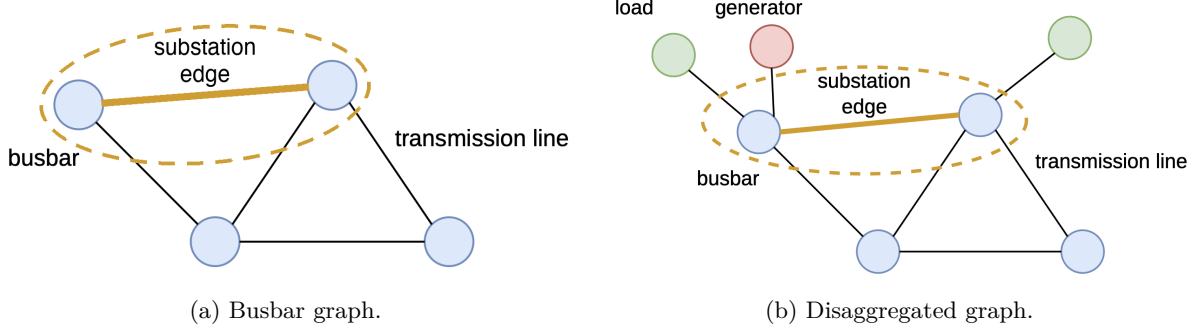


Figure 2.4: Direct same-substation busbar relation, shown in the two schemas of Section 2.2. The highlighted orange edge allows two busbars belonging to the same substation to exchange messages; it is a computational relation and not a physical transmission line. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes. The relation is identical in both cases: it connects two busbars of one substation and does not depend on the node types attached to them.

For a full graph containing S substations and L physical lines, the number of candidate directed edges is

$$|\mathcal{E}_{\text{cand}}| = 2B^2L \quad (2.26)$$

for the standard representation, and

$$|\mathcal{E}_{\text{cand}}| = 2B^2L + SB + SB(B - 1) \quad (2.27)$$

when self and same-substation relations are both enabled.

2.3.2 Substation summary nodes (n)

An optional hierarchical augmentation introduces one learned node u_s for every included substation s . Let $\mathcal{B}(s)$ be its included busbar nodes. **Summary edges are either bidirectional or toward_summary.** In the latter case, the augmented graph contains only

$$\mathcal{E}_{\text{sub}} = \{(i, u_s) \mid i \in \mathcal{B}(s)\}; \quad (2.28)$$

the bidirectional setting additionally includes every reverse edge (u_s, i) . Each substation node therefore gathers information from every busbar belonging to that substation. In the bidirectional case it can also redistribute its learned summary to those busbars in later message-passing layers. It is not connected directly to generators, loads, or transmission-line nodes; those elements influence it through their busbar attachments.

The initial substation-node states are learned embeddings indexed by the global substation identifiers. For S included substations and B busbars per substation, the augmentation adds S nodes and either SB directed edges for **toward_summary** or $2SB$ for **bidirectional**. The nodes participate in standard mean, sum, maximum, or attention readout. Controlled readout marks a substation node as controlled whenever at least one of its busbars is controlled. The sparse graph transformer assigns the attachments a dedicated learned relation.

This augmentation differs from the same-substation edges in Section 2.3.1: it introduces a learned hub u_s instead of directly connecting pairs of busbars. It can be combined with the virtual-node readout.

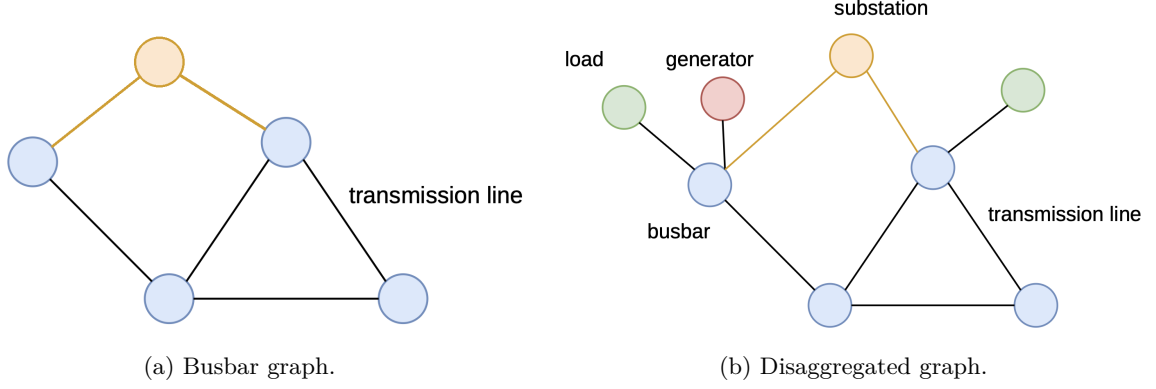


Figure 2.5: Learned substation-summary augmentation, shown in the two schemas of Section 2.2. The orange substation node receives messages from each of its blue busbar nodes and can optionally redistribute the resulting substation-level representation. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes; in both cases the summary node attaches only to busbars, so generators, loads, and transmission-line nodes reach it through their busbar attachments.

2.4 Pooling and Graph Readout

The final graph embedding is obtained either by pooling node embeddings directly or by reading out a learned virtual node. The output projection and action head are the same in both cases.

2.4.1 Mean, Max, Sum pooling

Mean pooling averages the final node embeddings over the valid nodes in the local graph:

$$\bar{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \quad (2.29)$$

where m_v is the valid-node mask. This is one mean over all valid node types, not one mean per type **to be invariant**. Thus the busbar graph pools busbar nodes, the disaggregated graph pools busbars, generators, and loads, and the disaggregated-line graph additionally pools transmission-line nodes. One-hop contextual nodes are included in the ordinary mean readout.

No sum pooling because ultimately the objective would be to have something transferable to other graph and sum pooling would be dependent on the number of nodes in the graph.

Maximum pooling instead takes an independent maximum in every feature channel:

$$\left(\bar{\mathbf{h}}_G^{\max}\right)_c = \max_{v: m_v=1} \left(\mathbf{h}_v^{(K)}\right)_c. \quad (2.30)$$

The three readouts differ in what they preserve. The mean is invariant to the number of pooled nodes, but it dilutes an isolated extreme node as the graph grows: one overloaded line among forty nodes moves the mean by a fortieth of its own deviation. The sum preserves that node’s full contribution and additionally encodes graph size, which the mean discards. The maximum is again invariant to the count and reports the strongest activation per channel, which suits detecting whether some node is in a critical state, at the cost of discarding how many nodes are in that state and of propagating gradients only to the arg-maximum node of each channel.

2.4.2 Virtual-node graph readout

A virtual-node readout replaces terminal mean, sum, maximum, or attention pooling with one learned graph-summary node. It can be combined with any of the three graph schemas.

Augmented graph

Let $v_\star$ denote the virtual node. Define the summary set $\mathcal{V}_{\text{sum}}$ as the included substation nodes when the augmentation from Section 2.3.2 is enabled, and as the included busbar nodes otherwise. The augmented graph is

$$\mathcal{V}^+ = \mathcal{V} \cup \{v_\star\}, \quad (2.31)$$

$$\mathcal{E}^+ = \mathcal{E} \cup \{(i, v_\star) \mid i \in \mathcal{V}_{\text{sum}}\}. \quad (2.32)$$

Equation 2.32 shows the `toward_summary` case; `bidirectional` additionally inserts $(v_\star, i)$ for every summary node. Thus the virtual node connects to all included busbars, or to all included substation-summary nodes when that augmentation is enabled. It has no direct edge to generator, load, or transmission-line nodes.

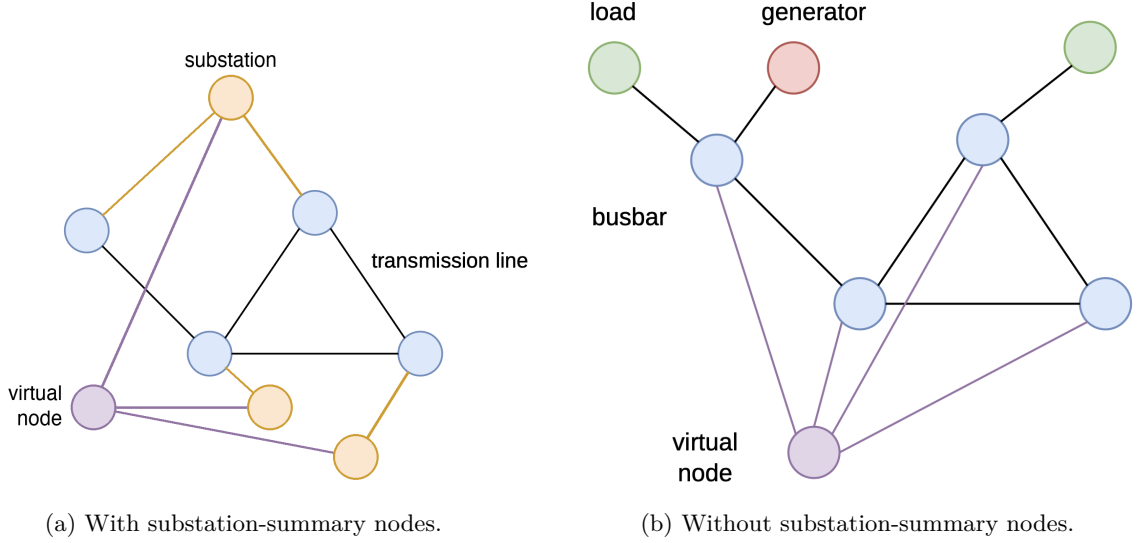


Figure 2.6: Virtual-node graph readout, shown for both definitions of the summary set $\mathcal{V}_{\text{sum}}$ in Equation 2.32. The purple node aggregates that set and its final embedding replaces terminal node pooling; reverse messages are optional. In (a) the substation-summary augmentation of Section 2.3.2 is enabled, so the virtual node attaches to the orange substation nodes and reaches the busbars only through them, which gives a two-level hierarchy busbar $\rightarrow$ substation $\rightarrow$ virtual node. In (b) the augmentation is disabled and the virtual node attaches directly to every included busbar. In neither case does it connect to generator, load, or transmission-line nodes.

Learned summary instead of terminal pooling

The initial virtual representation $\mathbf{h}_\star^{(0)}$ is learned. A message-passing layer jointly updates the physical and virtual nodes:

$$\left(\{\mathbf{h}_i^{(k+1)}\}_{i \in \mathcal{V}}, \mathbf{h}_\star^{(k+1)}\right) = \text{GNN}^{(k)}\left(\{\mathbf{h}_i^{(k)}\}_{i \in \mathcal{V}}, \mathbf{h}_\star^{(k)}, \mathcal{E}^+\right). \quad (2.33)$$

With `bidirectional` edges the virtual state can also influence its neighbors in later layers; with `toward_summary` it only aggregates. After K layers, the graph embedding is

$$\mathbf{z}_G = \text{ReLU}\left(\mathbf{W}_o \mathbf{h}_\star^{(K)} + \mathbf{b}_o\right), \quad (2.34)$$

rather than from $\text{Pool}(\{\mathbf{h}_i^{(K)}\}_{i \in \mathcal{V}})$. The output dimension is unchanged.

Relation attributes and encoder compatibility

Virtual connections are structural: their continuous edge attributes are zero, except for a neutral unit value in the `rho` position when that feature exists. The sparse graph transformer allocates dedicated learned relations for substation attachments and virtual-node attachments, allowing its attention layers

to distinguish hierarchy links from all physical and equipment relations.

Without substation nodes, a graph containing N_{bus} included busbars receives one virtual node and either N_{bus} or $2N_{\text{bus}}$ directed edges. With the hierarchy enabled, the virtual-node stage instead adds either S or $2S$ edges on top of the substation augmentation. After one layer the virtual node has aggregated its immediate busbar or substation neighbors.

2.5 Candidate-Action Scoring from the Explicit-Line Graph

The default action head maps the graph embedding to one output column per discrete action. That column is tied to an action index and not to the physical objects the action modifies. This section describes an alternative head that scores every candidate action from the graph nodes it touches. It is available only for the explicit-line schema of Section 2.2.3 and is selected with `actor_action_head=candidate_pool`; the default remains the action-index MLP head of Appendix A.5. The PPO objective, the categorical policy, sampling, deterministic evaluation, and checkpointing are unchanged.

2.5.1 Action-index head versus candidate scoring

With the default head, agent a produces its logit vector in one step from the readout $\mathbf{z}_a$ of Equation A.10:

$$\ell_a = \text{MLP}_a(\mathbf{z}_a), \quad \ell_a \in \mathbb{R}^{|\mathcal{A}_a|}. \quad (2.35)$$

The last layer of MLP_a has one row per action index, so the mapping from a grid state to a topology change must be learned separately for every index, and the head grows with $|\mathcal{A}_a|$.

Candidate scoring replaces Equation 2.35 by one shared scalar-valued network applied to every action:

$$\ell_{a,j} = \text{score} \left(\underbrace{\mathbf{z}_a}_{\text{global context}}, \underbrace{\mathbf{c}_j}_{\text{touched-node context}}, \underbrace{\phi_j}_{\text{static action features}} \right), \quad j \in \{0, \dots, |\mathcal{A}_a| - 1\}. \quad (2.36)$$

The scoring parameters are shared by all actions, so the head size no longer depends on $|\mathcal{A}_a|$, and the ranking of two actions is driven by the difference between their touched-node contexts rather than by two independent output rows. This makes the explicit line nodes directly usable: an action next to a loaded line is scored from that line’s own embedding.

2.5.2 Candidate-action metadata

The mapping from actions to node rows is static and is built once, when the environment and its graph specification are created, rather than at every rollout step. The explicit-line builder exports the row maps `busbar_id_to_node_row`, `line_id_to_node_row`, `gen_id_to_node_row`, `load_id_to_node_row`, and `substation_busbar_node_rows`, which give the node index of each physical object in the local graph of the agent and -1 for objects absent from that graph.

Each exposed action j of agent a is converted to its global Grid2Op action and decoded once. Bus-assignment and bus-change sections give the modified substations and, for each modified position of the topology vector, the object type and identifier: line endpoint, generator, or load. Line-status sections give the connected or disconnected lines; for those, both endpoint substations are added to the touched set, because a status change alters the local connectivity at both ends. This yields, for action j , the physical sets

$$\mathcal{S}(j), \quad \mathcal{L}(j), \quad \mathcal{G}(j), \quad \mathcal{D}(j) \quad (2.37)$$

of touched substations, lines, generators, and loads. They are converted to the node-index sets

$$\begin{aligned} \mathcal{N}_{\text{bus}}(j) &= \{i(s, b) \mid s \in \mathcal{S}(j), b \in \{0, \dots, B - 1\}\}, \\ \mathcal{N}_{\text{line}}(j) &= \{\text{node row of } \ell \mid \ell \in \mathcal{L}(j)\}, \end{aligned} \quad (2.38)$$

and analogously $\mathcal{N}_{\text{gen}}(j)$ and $\mathcal{N}_{\text{load}}(j)$. A touched substation therefore contributes all of its busbar nodes, not only the busbar currently used. The sets are stored as padded index matrices of shape $|\mathcal{A}_a| \times m_t$ with a boolean mask, where m_t is the largest number of touched objects of type t over the action space; padded entries hold index -1 and mask 0 .

The builder is strict. It rejects an action space whenever a touched line, generator, load, or substation is missing from the agent graph, whenever an index falls outside $[0, |\mathcal{V}|)$, whenever the exposed action 0 touches any object, or whenever a non-idle action touches no graph-represented object. Because a topology action at a controlled substation moves line endpoints whose line nodes may sit at the boundary of the local graph, the head requires `gnn_include_neighbors=true`. Reduced action spaces are supported: the metadata keeps the original action identifiers alongside the exposed ones, so the decoded physical context always refers to the action actually executed.

Each action also carries a static feature vector $\phi_j \in \mathbb{R}^{12}$, listed in Table 2.9. The five indicator entries are binary; the seven count entries are divided by their largest value over the action space of that agent, so all entries lie in $[0, 1]$ and are independent of the grid size.

Table 2.9: Static action-feature vector ϕ_j appended to each candidate score. Count entries are divided by the per-agent column maximum.

Group	Entries	Meaning
Indicators	<code>is_do_nothing</code> , <code>has_busbar_context</code> , <code>has_line</code> , <code>has_load</code> , <code>has_generator</code>	Whether the action is the idle action and which object types it touches
Scaled counts	<code>n_substations</code> , <code>n_topology_changes</code> , <code>n_line_status_changes</code> , <code>n_line_endpoint_changes</code> , <code>n_generator_changes</code> , <code>n_load_changes</code> , <code>n_other_changes</code>	Size of the action: how many substations and how many objects of each type it modifies

2.5.3 Node embeddings from the encoder

Candidate pooling needs the node states before readout. The encoders therefore expose

$$(\mathbf{z}_a, \mathbf{H}) = \text{encoder}_{\text{nodes}}(\mathcal{G}_t^{(a)}), \quad \mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d_h}, \quad (2.39)$$

where row v of $\mathbf{H}$ is the post-message-passing state $\mathbf{h}_v^{(K)}$ of Equation 2.29 and d_h is the hidden width, which is not the readout width of $\mathbf{z}_a$. The ordinary forward pass is a projection of Equation 2.39 onto its first output, so existing configurations produce identical embeddings. The GINE, GAT, GCN, and GraphSAGE encoders and the sparse graph transformer all provide it, and the flat-concatenation wrapper forwards it unchanged.

Only the $|\mathcal{V}|$ physical rows of Equation 2.17 are returned. Substation-summary nodes and the virtual node, when enabled, are appended after those rows and are excluded from candidate pooling; they still influence the physical rows through message passing and still determine $\mathbf{z}_a$ under virtual-node readout.

2.5.4 Typed candidate pooling

For each action j and each object type $t \in \{\text{bus, line, load, gen}\}$, the masked mean over the touched rows is

$$\mathbf{c}_j^{(t)} = \frac{1}{\max(1, |\mathcal{N}_t(j)|)} \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}. \quad (2.40)$$

The clamped denominator makes an empty type produce the zero vector rather than a division by zero, which is what happens for every type of the idle action. The two implemented pooling modes are

$$\begin{aligned} \text{typed_mean} : \quad \mathbf{c}_j &= [\mathbf{c}_j^{(\text{bus})}; \mathbf{c}_j^{(\text{line})}; \mathbf{c}_j^{(\text{load})}; \mathbf{c}_j^{(\text{gen})}] \in \mathbb{R}^{4d_h}, \\ \text{mean} : \quad \mathbf{c}_j &= \frac{1}{\max(1, |\mathcal{N}(j)|)} \sum_{v \in \mathcal{N}(j)} \mathbf{h}_v^{(K)} \in \mathbb{R}^{d_h}, \quad \mathcal{N}(j) = \bigcup_t \mathcal{N}_t(j). \end{aligned} \quad (2.41)$$

The typed mode keeps the roles separated, so the scorer can distinguish a stressed line node from the load attached to the same substation; the untyped mode is the smaller control. Attention pooling, in which the action queries its own touched nodes, is not implemented and is left for after the typed-mean baseline has been measured.

2.5.5 Shared scorer and idle action

All non-idle actions are scored by one network with a single output unit and the same hidden widths as the MLP head:

$$\ell_{a,j} = \text{MLP}_{\text{score}}([\mathbf{z}_a; \mathbf{c}_j; \phi_j]), \quad j \geq 1. \quad (2.42)$$

The action-feature block is omitted when `candidate_action_use_features=false`. Because the idle action touches no object, its pooled context is identically zero and would otherwise be scored by a network trained on non-zero contexts. It therefore receives a dedicated head over the global context only:

$$\ell_{a,0} = \text{MLP}_{\emptyset}(\mathbf{z}_a) + b_{\emptyset}, \quad (2.43)$$

with $b_{\emptyset}$ a learned scalar applied to action 0 alone. This makes the idle logit an explicit judgment of whether the local grid state is safe enough not to intervene. With `candidate_action_do_nothing_head=false`, action 0 instead passes through Equation 2.42 with $\mathbf{c}_0 = \mathbf{0}$ and keeps the bias $b_{\emptyset}$.

The initial idle prior is preserved. Setting `init_do_nothing_prob = p0` zeroes the output layer of $\text{MLP}_{\text{score}}$ and of $\text{MLP}_{\emptyset}$ and sets the remaining idle bias to

$$\log \frac{p_0 (|\mathcal{A}_a| - 1)}{1 - p_0}, \quad (2.44)$$

so the initial policy assigns probability exactly p_0 to action 0 and spreads the rest uniformly. The evaluation-time and training-time logit bonus on action 0 is added after Equations 2.42 and 2.43 and is therefore unaffected.

2.5.6 Configuration, cost, and current limits

Table 2.10 lists the flags. Construction fails unless the run also sets `actor_encoder=gnn`, `gnn_graph_type=heterogeneous_line`, and `gnn_include_neighbors=true`, which are the settings that make node embeddings and complete action metadata available. Setting `intervention_gate=true` is rejected as well: its two-level factorization would have to be combined with candidate pooling as a separate step. The metadata is attached to the agent graph specifications when the environment is built, so all workers and evaluation environments reuse the same static mapping.

Table 2.10: Configuration flags of the candidate-action head. Defaults reproduce the existing action-index head.

Flag	Values (default)	Effect
<code>actor_action_head</code>	<code>mlp</code> , <code>candidate_pool</code> (<code>mlp</code>)	Selects Equation 2.35 or Equation 2.36
<code>candidate_action_pool</code>	<code>mean</code> , <code>typed_mean</code> (<code>typed_mean</code>)	Pooling of Equation 2.41
<code>candidate_action_use_features</code>	<code>boolean</code> (<code>true</code>)	Appends ϕ_j to each candidate score
<code>candidate_action_do_nothing_head</code>	<code>boolean</code> (<code>true</code>)	Dedicated idle head of Equation 2.43

The candidate context is materialized as a tensor of shape $[\text{batch}, |\mathcal{A}_a|, d_c]$ with $d_c = \dim \mathbf{z}_a + 4d_h + \dim \phi_j$ under typed pooling, so memory grows with the action-space size. This is the reason to start on `bus14` and on reduced action spaces before the full WCCI action space.

Two properties matter for a fair comparison. First, the head no longer scales with $|\mathcal{A}_a|$, so a candidate-pool actor and an action-index actor with the same graph settings do not have the same parameter count; the comparison holds the encoder, PPO settings, reward, normalization, chronic split, and evaluation protocol fixed and reports both counts. Second, the idle logit is produced by a different mechanism than the other logits, so the action-0 frequency has to be compared against the action-index baseline rather than assumed comparable. The controlled comparison is one explicit-line configuration run twice, once with `actor_action_head=mlp` and once with `actor_action_head=candidate_pool`.

Correctness of the static mapping is checked by unit tests covering the exported row maps, the decoded touched objects of bus-change and line-status actions, the strict rejection of a local graph without neighbor context, finite logits and non-zero node-embedding gradients under both pooling modes, the exact idle prior of Equation 2.44, equality between the ordinary readout and the first output of Equation 2.39, and an actor-level sampling, evaluation, and checkpoint-reload pass.

Auxiliary heads over the same node embeddings, such as predicting which lines are about to overload, are a possible later addition; they are not part of the current head.

2.6 Graph Actor and Critic Configuration

The actor uses two message-passing layers with 128 hidden features. LayerNorm is applied after the input node projection and after each message-passing layer. The graph readout produces a 128-dimensional embedding, followed by an agent-specific action head with two 128-unit hidden layers. The graph encoder and output projection are shared across agents; the action heads are separate because their action-set sizes may differ. This is the default `actor_action_head=mlp` setting; the candidate-scoring alternative of Section 2.5 replaces only this head. The flat observation is not concatenated to the graph embedding, and learned global node-identifier embeddings are disabled. Local actors include one-hop neighboring context.

The centralized critic is a separate MLP over the normalized flat state and does not reuse the actor graph embedding. It is used only during centralized training. Graph-input normalization follows Section 2.1 and is independent of the critic’s flat-state normalization. The exact GINE and GAT updates, graph readout, output projection, and action-head equations are given in Appendix A.

Chapter 3

Graph-Design Screening

This chapter reports the graph-design screens. All are balanced full factorials on `bus14` with three seeds per cell, and all ask the same kind of question: does a specific choice in the actor of Chapter 2 change how well a decentralized MAPPO policy survives on held-out chronics?

- **Screen A** varies the depth and width of the actor encoder, which applies to every graph schema: a 3×4 grid, 36 runs. *Training at the time of writing.*
- **Screen B** varies the message-passing operator and the readout, GINE against GAT and mean against maximum pooling: a 2×2 factorial, 9 new runs plus one reused cell. *Training at the time of writing.*
- **Screen C** varies the three structural augmentations of the busbar graph – same-substation edges, substation-summary nodes, and virtual-node readout (Sections 2.3 and 2.4.2): a 2^3 factorial, 24 runs. *Complete.*
- **Screen D** varies the direction of the generator and load attachment relations in the disaggregated graph (Section 2.2.2): a 3×3 factorial, 27 runs. *Complete.*

Screens A, B, and C all run on the busbar graph and share one reference configuration, so they can be read as a sequence; only Screen D changes the schema. The two completed screens account for 51 finished training runs.

3.1 How This Chapter Is Organized

Each screen is presented as one block containing its design and its outcome together:

1. **Question.** Which design choice is uncertain?
2. **Design.** Which factors are varied, at which levels, and what is held fixed.
3. **Coverage.** How far the runs actually got, before any endpoint number is read.
4. **Result.** The measured tables.
5. **Reading.** What follows, and what does not.

3.2 Shared Protocol

Every screen uses the configuration in Table 3.1. Every setting not named as a screened factor is identical across all runs, so each screen is a paired comparison against its own baseline cell.

Table 3.1: Settings shared by all screens. Only the graph type and the screened factors differ between them.

Component	Fixed setting
Environment	bus14 , difficulty 0, topology actions, decentralized agents, normalized observations and rewards, 80/20 train/test chronic split with split seed 0
Actor	One GNN encoder shared by all agents; LayerNorm; node pre-encoder; two 128-unit action-head layers; one-hop neighbor context
Critic	Centralized MLP with three 256-unit hidden layers
Graph preprocessing	None: neither deterministic physical scaling nor running standardization
Training budget	15,000,000 environment steps, 72 parallel environments, 576 rollout steps
MAPPO update	10 epochs, 16 minibatches, actor and critic learning rates 10^{-4} with linear annealing, $\gamma = 0.9$, $\lambda = 0.95$, clip 0.2, target KL 0.02, entropy coefficient 0.01
Exploration	No initial do-nothing prior and no action-0 logit bonus
Sparse control	Disabled: no intervention gate, no intervention penalty, no evaluation-time rho heuristic
Evaluation	Deterministic evaluation every 82,944 environment steps, 10 episodes on each split
Seeds	0, 1, 2 in every cell

3.2.1 Reference configuration

Screens A, B, and C are all derived, seed by seed, from one reference configuration: the unaugmented busbar graph with a GINE encoder, two message-passing layers, 128 hidden and output features, and mean readout. It appears in the repository as `gs_s2_bus_n0_none_e0n0v0` and is the `e0n0v0` cell of Screen C, where its measured survival is reported (Section 3.5). Each of the three screens changes one group of settings relative to it and leaves everything else untouched, so the reference cell is the common origin of the whole busbar-graph part of this chapter. Screen D uses the disaggregated schema and has its own baseline cell.

3.2.2 Endpoint statistics

Survival is the percentage of an episode’s maximum length that the agent keeps the grid alive, reported on the test split. Each run saves the checkpoint with the highest test survival seen during training. That checkpoint is then re-evaluated deterministically over all 201 held-out test chronics.

3.2.3 Compute

The runs were executed on two clusters, JED (CPU) and IZAR (GPU); seeds of the same cell are spread across both, so the compute backend is not confounded with any screened factor.

3.3 Screen A: Encoder Depth and Width

Runs in progress. The design below is final; the coverage, result, and reading subsections are placeholders to be filled from the completed runs.

Question

The reference configuration uses two message-passing layers and 128 hidden features. Those values were fixed by convention rather than measured. Depth and width control different things: with K layers a node embedding can only see K hops away, and a local actor graph with one-hop context has a small diameter, so a third layer may add nothing that the readout can use; width sets the capacity of each node embedding and of the graph vector that the action head consumes. Is the default (2, 128) actor justified, is one hop enough, and is there a width below which the policy degrades?

-> We need to fix the hyperparameter for the message passing depth and the encoder width. This is the most general question in the chapter: depth and width apply to every schema, so the answer conditions how the later screens should be read.

Design

A complete 3×4 grid with three seeds per cell, 36 runs. Table 3.2 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the reference configuration of Section 3.2.1, seed by seed.

Table 3.2: Screen A. Screened parameters and the values tested. All other settings are held at Table 3.1.

Parameter	Setting	Values tested
Message-passing layers K	gnn_layers	1, 2, 3
Encoder width d_h	gnn_hidden_dim	16, 32, 64, 128

Coverage

[pending: completed steps per run, common budget, and whether any cell lost a seed. Small-width runs are cheaper per step, so the walltime limit is expected to bind unevenly across the width axis — if it does, report the common budget per width column rather than one budget for the whole grid.]

Result

Table 3.3: Screen A. Test survival by message-passing depth and encoder width, averaged over three seeds, with the seed-to-seed standard deviation in parentheses. *[pending: fill from the completed runs; state which endpoint statistic of Section 3.2.2 is used.]*

Layers K	Encoder width d_h			
	16	32	64	128
1	—	—	—	—
2	—	—	—	—
3	—	—	—	—

Table 3.4: Screen A. Replication check: the (2, 128) cell against the e0n0v0 result of Screen C, which is the same configuration launched separately. *[pending: fill; a gap here is run-to-run noise, not an effect.]*

Launch	Mean (%)	Std (pp)	Seeds
Screen C e0n0v0	97.31	1.22	3
Screen A (K, d_h) = (2, 128)	—	—	3
Difference	—		

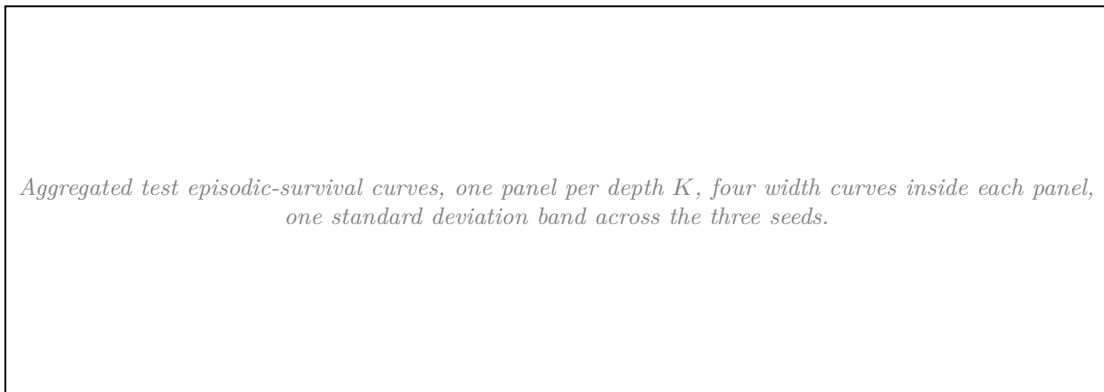


Figure 3.1: Screen A. Survival curves by depth and width. *[pending: replace the box with the exported figure.]*

Depth $\times$ width heatmap of the endpoint statistic, and the same heatmap expressed as a difference from the (2, 128) cell.

Figure 3.2: Screen A. Depth-by-width heatmap. *[pending: replace the box with the exported figure.]*

Actor parameter count against survival, one marker per cell, so a cell that matches the reference at a fraction of the parameters is visible directly.

Figure 3.3: Screen A. Cost against survival. *[pending: replace the box with the exported figure.]*

Reading

[pending: answer four questions explicitly.] First, whether the $K = 1$ row matches $K = 2$: if it does, one hop of message passing is sufficient on a local bus14 graph and the second layer is paying for nothing. Second, whether $K = 3$ recovers, matches, or degrades relative to $K = 2$, which bears directly on the Screen C finding that the hierarchy augmentations fail with only two layers (Section 3.5). Third, whether there is a width threshold: a monotone width effect and a cliff between two adjacent widths lead to different conclusions about what the graph vector has to carry. Fourth, whether the replication check in Table 3.4 is small relative to the differences claimed in Screens C and D; if it is not, the margins in those screens need re-reading with the wider error bar.

3.4 Screen B: Encoder Operator and Readout

Runs in progress. The design below is final; the coverage, result, and reading subsections are placeholders to be filled from the completed runs.

Question

Two independent choices have been held fixed at their defaults. The message-passing operator can be GINE, which adds the projected edge vector to the sender state inside every message, or GAT, which uses the edge vector to weight how strongly each incoming message counts (Appendix A). The readout can be the mean over valid nodes or a per-channel maximum (Section 2.4.1). The two interact in principle: a softmax-weighted aggregation followed by a maximum readout selects twice, while mean pooling after an unweighted sum preserves how many nodes are in a given state. Which combination suits the busbar graph?

The busbar schema is the relevant setting for a second reason. Its edge vector carries only the physical line block and no relation one-hot (Table A.3), so GAT’s attention here is computed from line state — loading, status, cooldown — rather than from relation identity. If attention helps anywhere, a graph whose edges differ only by physical measurement is where it should show.

Design

A 2×2 factorial over encoder $\in \{\text{GINE}, \text{GAT}\}$ and readout $\in \{\text{mean}, \text{max}\}$ with three seeds per cell. Only three cells are newly launched — 9 runs in `configs/gnn_graph_screening/stage3_encoder_pooling` — because the GINE/mean cell is the reference configuration of Section 3.2.1 and is reused rather than repeated. GAT uses four heads averaged rather than concatenated, so all four cells return the same 128 features. Every other setting is inherited from the reference configuration seed by seed.

Reusing a cell has a cost that must be carried into the reading: the GINE/mean numbers come from a different launch than the other three, so any difference against them contains whatever run-to-run drift exists between launches. That drift is exactly what Screen A’s replication check measures (Table 3.4), so the two screens should be read together.

Coverage

[pending: completed steps per run and the common budget. GAT is more expensive per step than GINE at equal width, so the two encoder rows may not reach the same step count under a shared walltime limit; if they do not, compare at the common budget and say so rather than at each run’s own endpoint.]

Result

Table 3.5: Screen B. Test survival by encoder and readout, averaged over three seeds, with the seed-to-seed standard deviation in parentheses. The GINE/mean cell is inherited from the reference configuration. *[pending: fill the three new cells using the same endpoint statistic as the inherited cell.]*

Encoder	Readout	
	Mean	Max
GINE	97.31 (1.22) [†]	–
GAT	–	–

[†]Reference configuration, full-test survival of the best checkpoint; not a new run.

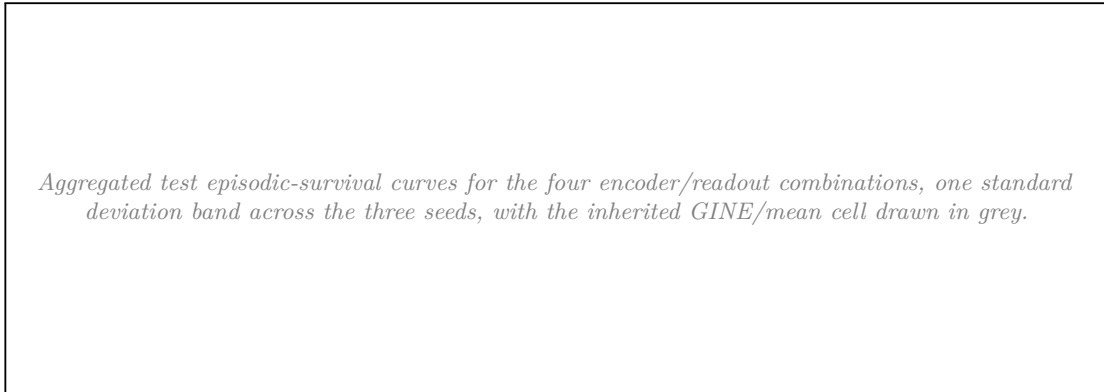


Figure 3.4: Screen B. Survival curves by encoder and readout. *[pending: replace the box with the exported figure.]*

Two panels: readout compared within each encoder, and encoder compared within each readout. If the same readout wins in both encoder panels, the two factors are acting independently.

Figure 3.5: Screen B. Factors compared within each other. *[pending: replace the box with the exported figure.]*

Reading

[pending: report the four cells, then answer three questions.] Whether either factor has a main effect large relative to the seed spread; whether they interact, which the 2×2 can show but not explain; and whether any gain from GAT is worth its cost per step, since a policy that reaches the same survival with a cheaper operator is the better default for the larger grids. State explicitly whether the inherited GINE/mean cell was re-run for confirmation, and if it was not, bound the comparison with the replication gap from Screen A.

3.5 Screen C: Structural Augmentations of the Busbar Graph

Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges (e), which let the busbars of one substation exchange messages directly; substation-summary nodes (n), which insert one learned hub per substation; and virtual-node readout (v), which replaces mean pooling with a learned graph-summary node. Each is motivated by the same intuition — shorten the path between related busbars — so the question is whether any of them earns its cost, and whether they compose.

Design

A complete 2^3 factorial with three seeds per cell, 24 runs, in `configs/gnn_graph_screening/stage2_structure`. The graph type is `bus` throughout and the baseline cell is the unaugmented `e0n0v0`, which is the reference configuration of Section 3.2.1. Cells are named by their three switches, so `e1n0v1` means same-substation edges on, summary nodes off, virtual-node readout on. Because the design is a cube rather than a grid, the eight cells are reported in full before any marginal.

Coverage

All 24 runs finished, each reaching at least 99.2% of the 15M-step budget. Endpoint numbers come from the full-test statistic, so each cell aggregates three runs $\times$ 201 chronics. The saved best checkpoint is late in training for most runs, between 13.7M and 14.9M steps, but not for all: the `e0n1v0` seed-1 checkpoint was selected at 2.65M steps and the `e1n1v1` seed-2 checkpoint at 9.87M, which is a symptom of those runs never improving on an early peak.

Result

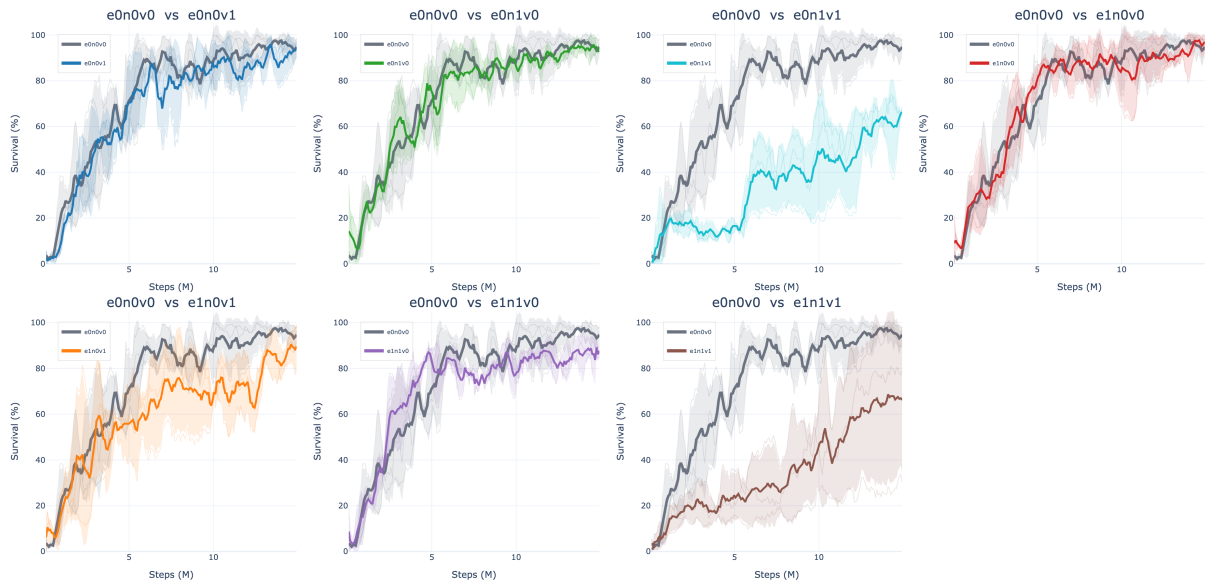


Figure 3.6: Screen C. Test episodic survival during training, one panel per augmented cell against the unaugmented $e0n0v0$ baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

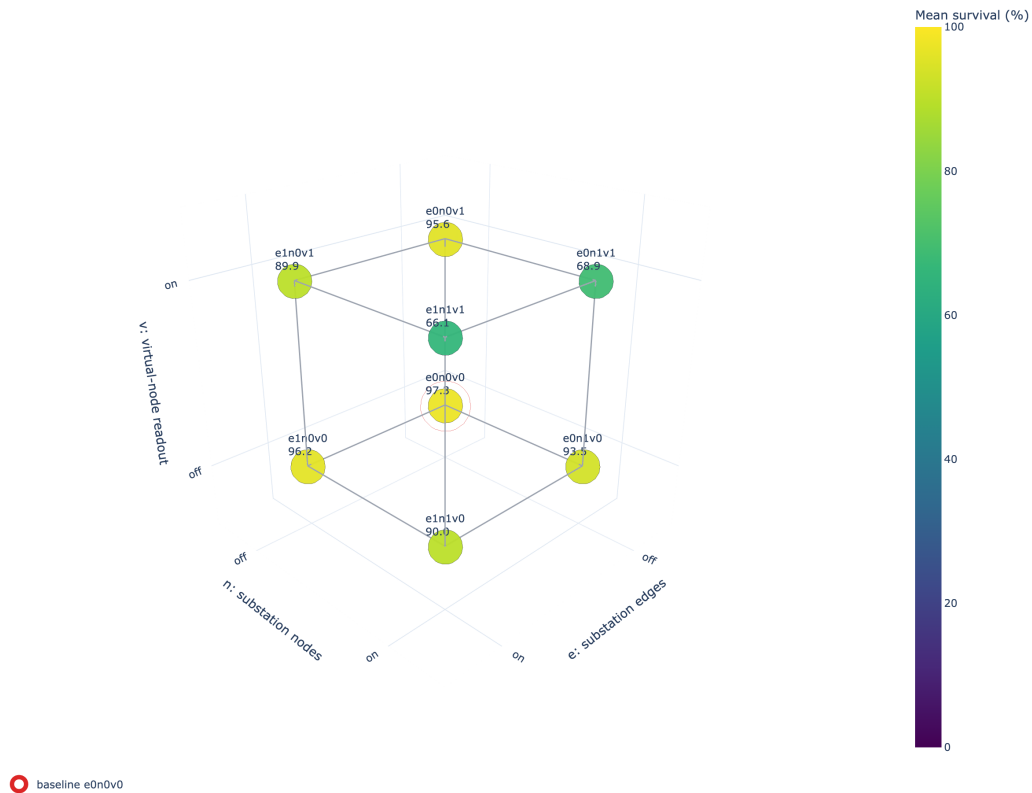


Figure 3.7: Screen C. The factorial as a cube: each vertex is one of the eight cells and each edge flips a single factor with the other two held fixed. The baseline $e0n0v0$ is circled in red.



Figure 3.8: Screen C. The same eight cells as a heatmap of the full-test survival of each run’s best checkpoint, with the three individual seed values printed under each cell mean.

Table 3.6: Screen C. Cells ranked by full-test survival, with the difference from the unaugmented **e0n0v0** baseline.

Cell	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
e0n0v0	97.31	1.22	96.55	98.72	0.00
e1n0v0	96.20	3.94	91.66	98.68	−1.11
e0n0v1	95.63	6.19	88.52	99.82	−1.68
e0n1v0	93.51	6.39	86.72	99.40	−3.80
e1n1v0	90.01	4.92	85.42	95.19	−7.30
e1n0v1	89.92	9.42	80.48	99.33	−7.39
e0n1v1	68.91	5.77	62.59	73.91	−28.40
e1n1v1	66.11	33.95	27.86	92.68	−31.20

Table 3.7: Screen C. Main effect of each factor (12 runs per level) and the four single-factor flips of the cube, each one enabling that factor with the other two held fixed. **Spread** is the range of those four flips.

Factor	Off (%)	On (%)	Effect (pp)	Flips (pp)	Mean flip (pp)	Spread (pp)
<i>e</i> (substation edges)	88.84	85.56	−3.28	−1.1, −5.7, −3.5, −2.8	−3.28	4.60
<i>v</i> (virtual readout)	94.26	80.14	−14.11	−1.7, −24.6, −6.3, −23.9	−14.11	22.92
<i>n</i> (summary nodes)	94.77	79.63	−15.13	−3.8, −26.7, −6.2, −23.8	−15.13	22.92

Reading

No augmentation earns its place, and the result is unambiguous: the unaugmented **e0n0v0** baseline is the best cell at 97.31%, and all twelve single-factor flips in Table 3.7 are negative. Not one of the three

mechanisms helps in any context. The baseline is also the most stable cell in the screen, with a seed spread of 1.22 points against 3.9–33.9 for every augmented cell, so the augmentations cost reliability as well as survival. The training curves in Figure 3.6 separate the cells well before the budget ends, so this is not an artefact of where the runs were cut.

The two hierarchy factors dominate, at -15.13 points for summary nodes and -14.11 for virtual-node readout, and they are not independent: each is mild in the cell where the other is off (n costs 3.80 points at **e0v0**; v costs 1.68 at **e0n0**) and severe once the other is on, with both **n1v1** cells landing between 66% and 69%. The upper face of the cube in Figure 3.7 shows this directly, and the spread column of Table 3.7 makes it explicit: for n and v the four flips range over 23 percentage points, so the main effect alone is not a usable summary of either factor.

The interpretation is that the two hierarchy mechanisms are redundant rather than complementary. Both insert a learned aggregation node above the busbars, and the virtual node attaches to the summary nodes when they exist (Figure 2.6), so enabling both stacks two learned hubs in series between the busbars and the readout. With only two message-passing layers, the readout of the **n1v1** cells is separated from most physical nodes by more hops than the encoder has layers. Screen A varies exactly that depth, so its $K = 3$ row is the direct test of this explanation.

Same-substation edges are the mildest of the three, at -3.28 points on average and within a narrow band: its four flips range only from -1.11 to -5.71 , so unlike n and v it behaves like a small, consistent cost rather than a context-dependent one. Its best cell **e1n0v0** sits 1.11 points below the baseline, which is inside the seed spread of both cells, and the per-seed values in Figure 3.8 show why nothing stronger can be claimed: two of its seeds reach 98.7% and 98.3% while the third reaches only 91.7%, straddling the baseline’s much tighter 96.6–98.7%. The honest reading is that the edges neither help nor clearly hurt on their own, and that they add candidate edges for no measured return.

The operational conclusion is to keep the plain busbar graph with mean readout for subsequent stages, and to treat any future hierarchy proposal as needing a deeper encoder rather than another augmentation on top of two layers.

3.6 Screen D: Generator and Load Message Direction

Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 2.2.2 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the mean readout of Section 2.4.1 pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

This is the one screen that leaves the busbar schema, so it has its own baseline cell rather than the reference configuration of Section 3.2.1.

Design

A complete 3×3 factorial over the generator and load directions with three seeds per cell: 27 runs, held in the configuration folder `configs/gnn_graph_screening/heterogenous_message_direction`. The graph type is **heterogeneous** throughout; the line and summary directions stay **bidirectional** and are inert for this schema. The encoder matches the reference configuration — GINE, two layers, 128 features, mean readout, no augmentations — so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated **bi** (bidirectional), **a2b** (asset to busbar), and **b2a** (busbar to asset).

Coverage

All 27 runs finished. Every run reached at least 96% of the 15M-step budget and most reached 14.97M steps; the common budget is 14.432M steps, set by `gbi_1a2b_s1`. Endpoint numbers come from the full-test statistic, so each cell aggregates three runs $\times$ 201 chronics. The saved best checkpoint is not

always late in training: for one seed of the a2b/a2b cell it was selected at 4.31M steps, which is itself a symptom of that cell's instability.

Result

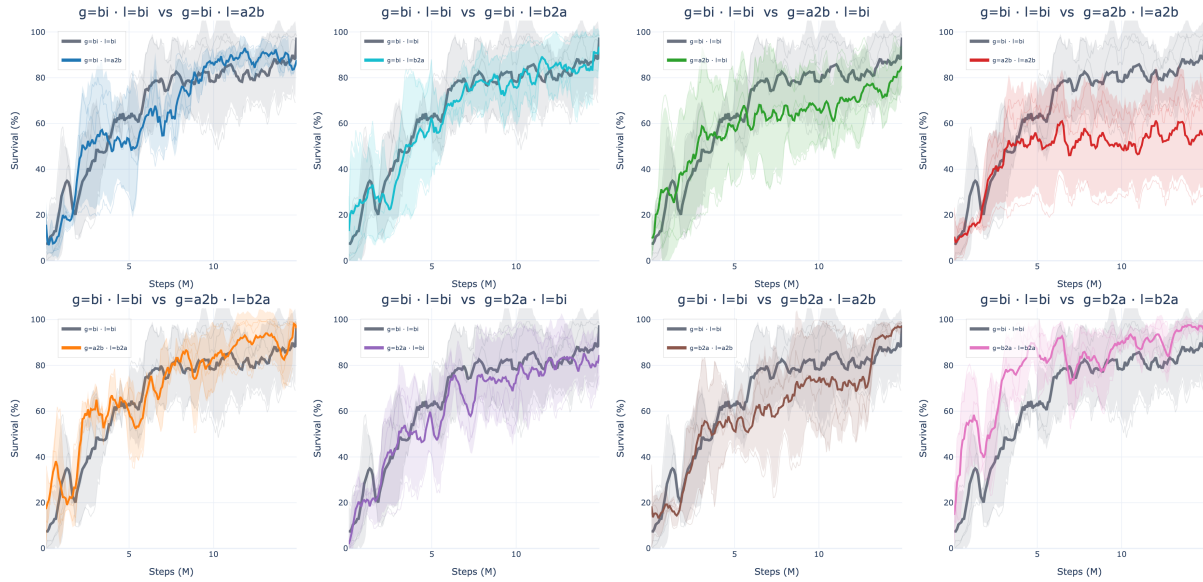


Figure 3.9: Screen D. Test episodic survival during training, one panel per direction pair against the bidirectional baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

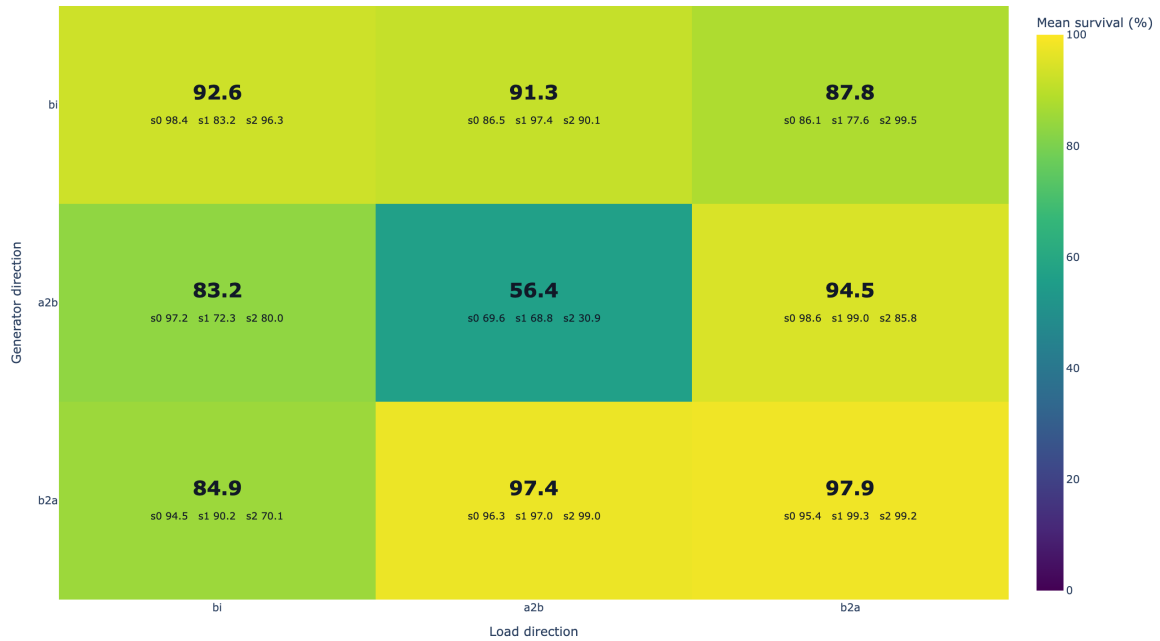


Figure 3.10: Screen D. Full-test survival of the best checkpoint by generator direction (rows) and load direction (columns), with the three individual seed values printed under each cell mean. The baseline cell is bi/bi.

Table 3.8: Screen D. Direction pairs ranked by full-test survival, with the difference from the **bi/bi** baseline.

Generator / load	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
b2a / b2a	97.94	2.20	95.40	99.25	+5.30
b2a / a2b	97.43	1.42	96.32	99.03	+4.79
a2b / b2a	94.47	7.53	85.78	99.03	+1.83
bi / bi	92.65	8.24	83.21	98.40	0.00
bi / a2b	91.35	5.54	86.52	97.40	-1.30
bi / b2a	87.76	11.04	77.64	99.53	-4.89
b2a / bi	84.93	13.01	70.11	94.46	-7.72
a2b / bi	83.16	12.75	72.27	97.19	-9.49
a2b / a2b	56.41	22.12	30.88	69.60	-36.24

Reading

Busbar-to-asset is the better direction for both relations. Averaging over the other relation, the generator marginal is 93.43% for **b2a**, 90.58% for **bi**, and 78.01% for **a2b**; the load marginal follows the same order at 93.39%, 86.91%, and 81.73%. The strongest part of the result is not the mean but the spread: the two best cells, **b2a/b2a** and **b2a/a2b**, have seed standard deviations of 2.20 and 1.42 percentage points against 8.24 for the bidirectional baseline, and their worst seeds (95.40% and 96.32%) are above the baseline’s mean. The seed values printed in Figure 3.10 show this at a glance: the bottom row holds three tightly grouped seeds per cell, while the baseline cell spans 83.2% to 98.4%. A gain of about five points that comes with a fourfold reduction in seed spread is a different kind of finding from a gain of five points at unchanged variance.

The failure case is equally clear. Making both relations asset-to-busbar collapses the policy to 56.41% with a 22.12-point spread and a worst seed at 30.88%, which is 36 points below the baseline and the largest single effect measured anywhere in this chapter. Its panel in Figure 3.9 separates from the baseline within the first two million steps and never recovers, so this is a training failure rather than a late collapse.

A plausible mechanism, consistent with the readout defined in Section 2.4.1 but not directly measured here, is that the mean pooling includes generator and load nodes. Under **b2a** an asset node receives busbar context and its updated embedding enters the readout, so the asset row carries information about its local neighborhood. Under **a2b** the asset node never receives a message, so after K layers its embedding is still a projection of its own raw features, and it dilutes the pooled vector with context-free rows; with both relations set that way, every generator and load row is inert. Testing this would require re-running the worst cell with controlled readout, which restricts pooling to controlled nodes, and is not part of the present screen.

Two caveats bound the conclusion. It is measured on **bus14** only, and the full-test statistic reports selected checkpoints, so the level of every cell is optimistic even though the comparison between cells is paired.

3.7 Campaign Status

Table 3.9 lists every screening folder that exists in the repository, so that configured work is not read as measured work. Only Screens C and D carry results; Screens A and B are training.

Table 3.9: State of the graph-screening campaign. “Complete” means a finished factorial with a seed-aggregated summary in this chapter.

Configuration folder	Runs	Status
<code>stage3_depth_width</code>	36	Training: Screen A
<code>stage3_encoder_pooling</code>	9	Training: Screen B
<code>stage2_structure</code>	24	Complete: Screen C
<code>heterogenous_message_direction</code>	27	Complete: Screen D
<code>stage1_representation_normalization</code>	12	Run under an earlier explicit-line edge definition; superseded
<code>stage1b_normalization_confirmation</code>	9	Superseded by the same change
<code>stage1c_provisional_structure</code>	7	Provisional; replaced by Screen C
<code>stage1d_heterogeneous_directions</code>	8	Provisional; replaced by Screen D
<code>stage1e_heterogeneous_structure</code>	8	Not analysed
<code>stabilize_training</code>	6	Configured, not analysed
<code>stage3_encoder</code>	0	Placeholder, no configurations
<code>stage4_confirmation</code>	0	Placeholder, no configurations

The two completed screens fix two defaults for the remaining stages: the busbar graph is used without structural augmentation, and any run on the disaggregated graph attaches generators and loads with busbar-to-asset relations rather than the bidirectional default. Both defaults are established on **bus14** alone and would need re-checking before being carried to a larger grid. Screens A and B may revise the encoder that both defaults were measured with, in which case the two completed screens should be spot-checked at the revised encoder rather than assumed to carry over.

Chapter 4

Experimental Results

4.1 How to Read This Chapter

This chapter is organized as a sequence of experiment blocks. The goal is to make the thesis easy to navigate while many tests are still being added. Each block follows the same structure:

1. **Question.** What uncertainty or design choice does this test address?
2. **Baseline and change.** Which run is the control, and what is changed in the comparison runs?
3. **Protocol.** Which data split, checkpoint, seeds, and metric are used?
4. **Implementation detail.** Which algorithmic, architectural, or mathematical change is being tested, when this is relevant?
5. **Evidence.** Which table and figure summarize the result?
6. **Interpretation.** What can be concluded, and what still needs to be verified?

Using the same block structure for every new experiment should make it possible to add future MSc-thesis tests without rewriting the whole results chapter.

4.2 Evaluation Protocol

The experiments use a train/test chronic split. The training set contains 80% of the chronics and is used to update the policy. The test set contains the remaining 20% and is used to estimate generalization during training and for final evaluation.

During training, evaluation rollouts are periodically run on both the train and test splits using 10 episodes each time. In the current rerun notebook, the main reported metric is **test episodic survival**, expressed as a percentage. Higher survival means that the agent keeps the grid alive for a larger fraction of the evaluation episodes. The final plots show seed-aggregated mean curves, faint individual seed curves, and the variability band.

The final evaluation currently uses the saved checkpoint with the best combined train/test survival. This selection rule should be reported explicitly because the test split participates in checkpoint selection; the values in this chapter are therefore best interpreted as controlled rerun comparisons, not as a final untouched test-set estimate.

4.3 Experiment Index

Table 4.1 gives a compact map of the experiment blocks included so far. New result sections should add one row to this index before adding detailed prose.

Table 4.1: Current result blocks and their role in the thesis.

Block	Main question	Primary evidence	Status
A0 core reruns	Which baseline training changes stabilize MAPPO?	Table 4.3, Figures 4.1 and 4.2	Drafted
Initial do-nothing bias	Does an initial action-0 prior help exploration?	Table 4.4, Figure 4.3	Drafted
GINE reruns	Do graph encoders improve survival over the A0 baseline?	Tables 4.5 and 4.6, Figure 4.4	Drafted
Heavy GINE	Does a larger GINE architecture help or overcomplicate optimization?	Table 4.7, Figure 4.5	Drafted
Sparse intervention control	Can decentralized agents intervene rarely, locally, and only when useful?	Tables 4.9 and 4.10	Design drafted

4.4 IEEE 14-Bus Experiments

The first result group focuses on the IEEE 14-bus topology-control task. The initial PPO/MAPPO baseline from the MARL2Grid framework was not satisfactory: the agents did not learn a robust policy and the survival rate remained low. The following experiments therefore test training and architecture changes that could make the baseline more stable.

4.4.1 A0 Core Reruns

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. The A0 family starts from the flat MLP MAPPO baseline and tests three main stabilization choices:

- increasing actor and critic capacity from two hidden layers to three hidden layers;
- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent.

Implementation detail. There is no new mathematical objective in this ablation. The implementation changes are practical training choices: network depth, reward normalization, critic-update scheduling, and the initialization of the action-0 probability. Table 4.2 records these settings so that the survival plots can be interpreted without repeatedly restating the run names.

Evidence. Table 4.3 reports final and peak survival for the core A0 ablations. Figure 4.1 shows each comparison against the `rerun_a0_opt` baseline, and Figure 4.2 overlays all core A0 curves.

Interpretation. The optimized A0 baseline, the non-optimized critic-update variant, and the smaller MLP variant all finish around 93% survival. This suggests that the main A0 recipe is robust once the rerun protocol is fixed. Disabling reward normalization still reaches high peak survival, but it learns more slowly and finishes lower. The old no-validation, non-optimized baseline is clearly not competitive and should not be used as the main reference for later ablations.

Table 4.2: A0-family run configurations. The reference run is `rerun_a0_opt`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from <code>rerun_a0_opt</code>
<code>rerun_a0_opt</code>	$3 \times 256 / 3 \times 256$	true	0.0	true	Reference baseline
<code>rerun_a0_nonopt</code>	$3 \times 256 / 3 \times 256$	true	0.0	false	Disables optimized critic updates
<code>rerun_a01</code>	$2 \times 256 / 2 \times 256$	true	0.0	true	Uses a smaller MLP actor and critic
<code>rerun_a02</code>	$3 \times 256 / 3 \times 256$	false	0.0	true	Disables reward normalization
<code>old_a0_nonopt_nova120</code>	$3 \times 256 / 3 \times 256$	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
<code>rerun_bias_05</code>	$3 \times 256 / 3 \times 256$	true	0.5	true	Uses action-0 bias 0.5
<code>rerun_bias_07</code>	$3 \times 256 / 3 \times 256$	true	0.7	true	Uses action-0 bias 0.7

A0 core reruns: `rerun_a0_opt` baseline pairwise comparisons

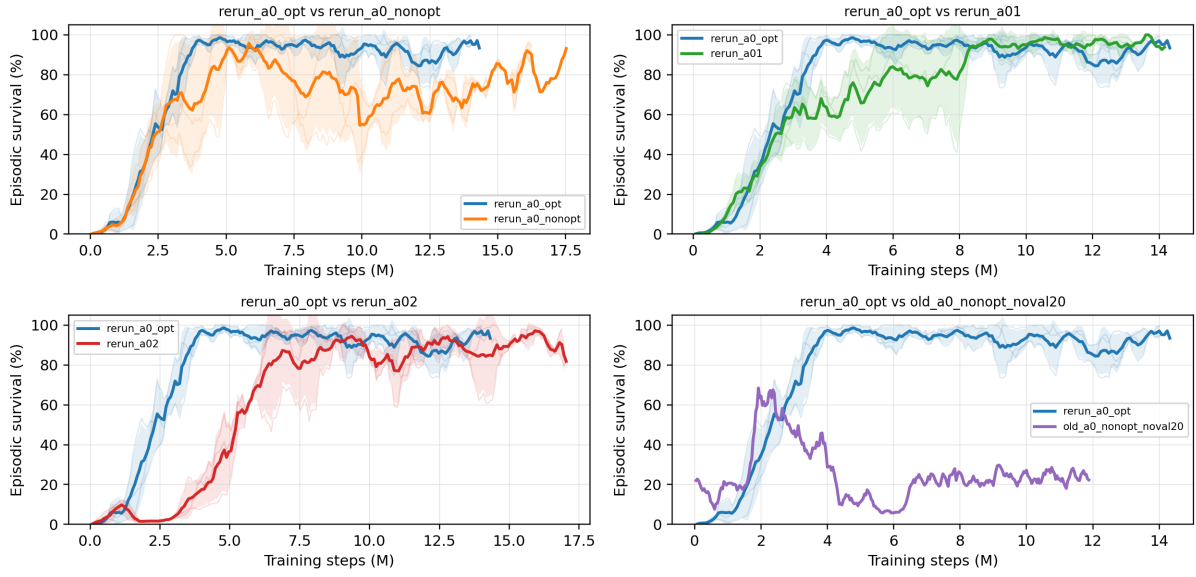


Figure 4.1: Pairwise A0 rerun comparisons against the `rerun_a0_opt` baseline.

A0 core reruns: rerun_a0_opt vs rerun_a0_nonopt / rerun_a01 / rerun_a02 / old noval20 nonopt

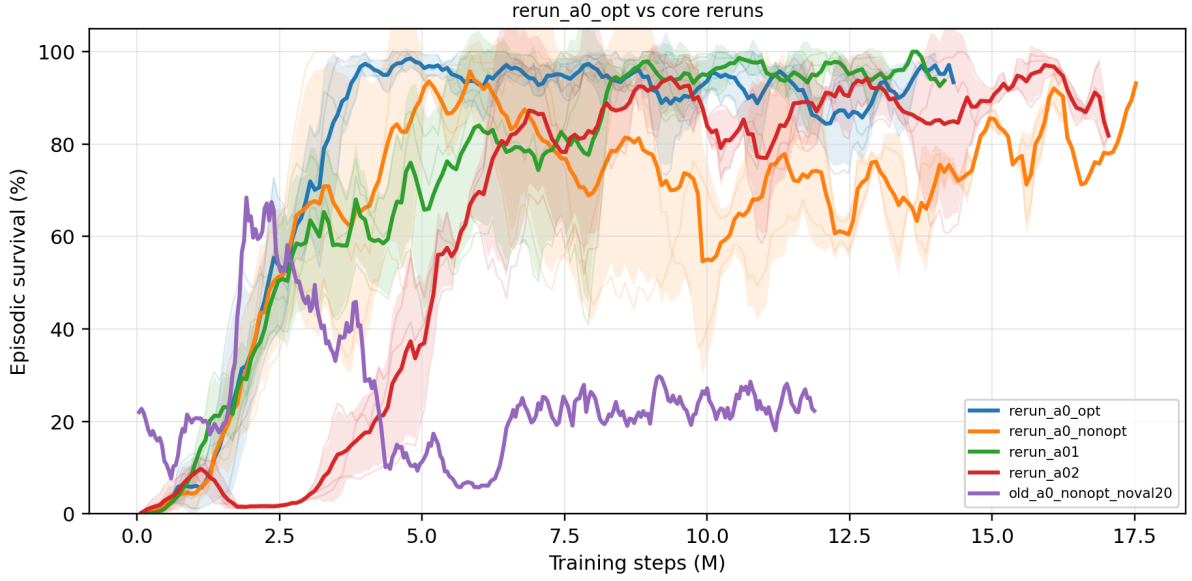


Figure 4.2: All A0 core rerun survival curves shown on one axis.

Table 4.3: A0 core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
rerun_a0_opt	A0 optimized baseline	93.4	98.6
rerun_a0_nonopt	Disable optimized critic update	93.1	95.8
rerun_a01	Smaller MLP actor/critic	93.7	100.0
rerun_a02	Disable reward normalization	81.8	97.1
old_a0_nonopt_noval20	Older non-optimized protocol	22.3	68.4

4.4.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning?

Baseline and changes. The A0 baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 4.4 and Figure 4.3 compare the zero-bias A0 baseline with rerun_bias_05 and rerun_bias_07.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is therefore not that larger bias is always better. Instead, the initialization prior appears to strongly shape exploration and should be treated as a first-order ablation variable.

Table 4.4: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
<code>rerun_a0_opt</code>	93.4	98.6
<code>rerun_bias_05</code>	50.8	70.2
<code>rerun_bias_07</code>	99.3	100.0

`rerun_a0_opt` baseline vs `rerun_bias_05` / `rerun_bias_07`

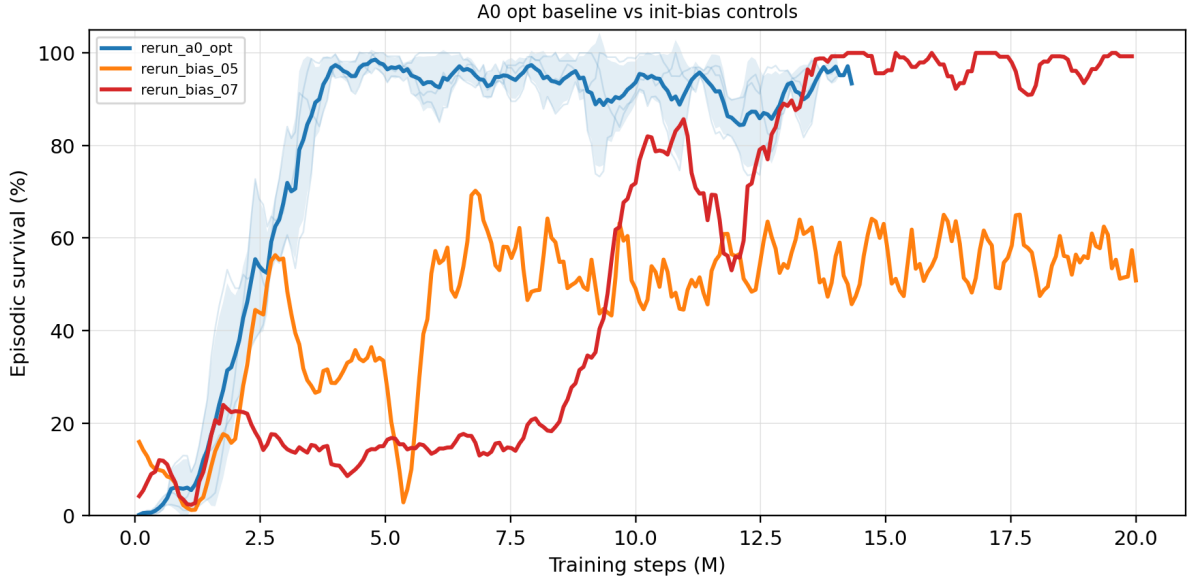


Figure 4.3: A0 baseline compared with initial do-nothing bias controls `rerun_bias_05` and `rerun_bias_07`.

4.4.3 GINE Encoder Reruns

Question. Can graph encoders improve topology-control performance, and which GINE design choices matter most?

Baseline and changes. The comparison starts from the historical `best_00` GINE baseline. The reruns then test lighter GINE encoders, MLP versus GNN critics, optimized critic updates, flat-feature concatenation, A0-style entropy, and different do-nothing-bias settings.

Protocol. All GINE runs use `gnn_type = gine`, reward normalization, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the A0 experiments. The reference run is `best_00`.

Implementation detail. The complete GINE/GAT graph-actor pipeline, including edge-conditioned message passing, pooling, and the action head, is described in Appendix A. This block separates representational changes from optimization changes. The GINE encoder changes how grid observations are embedded, while the critic type, flat-feature concatenation, entropy setting, and critic-update schedule affect how the PPO objective is optimized.

Evidence. Table 4.5 records the run configurations, Table 4.6 gives the selected survival values, and Figure 4.4 shows the pairwise survival curves against `best_00`.

Interpretation. The strongest GINE result is `best_14`, which reaches about 95.7% final survival. This suggests that the graph encoder can be competitive when it is paired with a simpler MLP critic and stable A0-style training choices. The heavier historical GINE baseline has high peak survival but much weaker final survival, which points to optimization stability rather than representational capacity as the main issue.

Table 4.5: GINE rerun configuration summary. The reference run is **best_00**.

Run	Actor/critic encoder	GNN hid- den/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from best_00
best_00	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
best_10	gnn / mlp	64 / 1	$2 \times 128 / 2 \times 128$	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
best_11	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
best_12	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
best_13	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.01	0.0	true	A0-style heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
best_14	gnn / mlp	64 / 1	$2 \times 128 / 2 \times 128$	false	false	0.01	0.0	true	Light A0-style GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Table 4.6: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
best_00	Historical GINE baseline	59.2	93.9
best_10	Light GINE with MLP critic	78.9	90.4
best_12	Optimized critic and bias 0.0	82.5	85.7
best_14	Light A0-style GINE with MLP critic	95.7	97.4

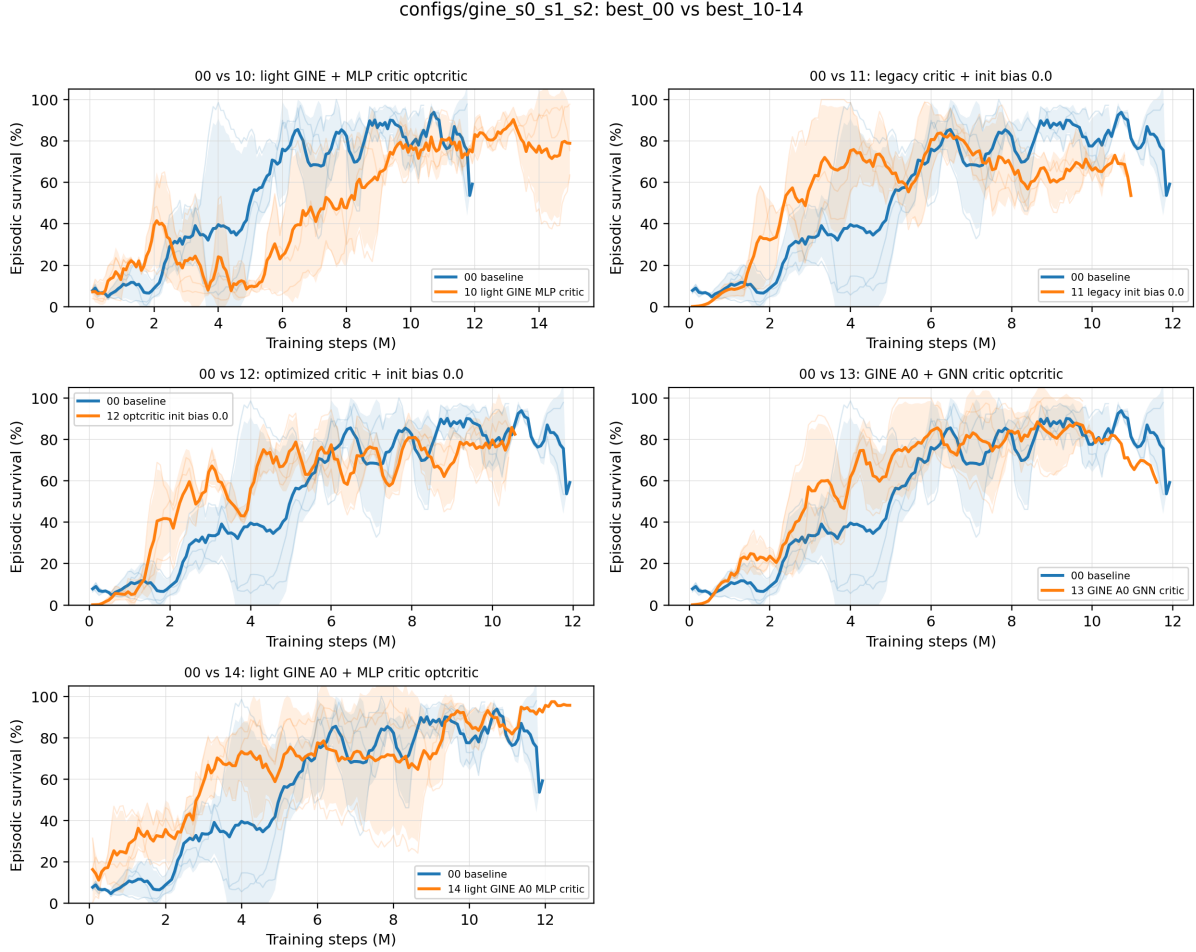


Figure 4.4: Seed-aggregated test episodic survival for GINE **best_00** against **best_10–best_14**.

4.4.4 Heavy GINE

Question. Does the heavier concat-flat GINE baseline fail because it lacks capacity, or because specific architectural and optimization choices make it unstable?

Baseline and changes. The reference run is **best_00**, the historical shared-actor, concat-flat GINE with a GNN critic and legacy critic updates. The best-search 2 family compares this baseline against controlled alternatives that change the flat-feature concatenation, critic type, actor sharing, GNN size, entropy schedule, node identifiers, message-passing operator, and initialization bias.

Protocol. The configurations are read from `configs/gine_s0_s1_s2`. Each family is run with seeds `s0`, `s1`, and `s2`. Unless changed by the family, the runs use reward normalization, deterministic evaluation, 5 evaluation episodes, 15,000,000 total timesteps, evaluation every 80,000 timesteps, and the same 80/20 chronic split.

Implementation detail. Table 4.7 gives the parameter comparison used for this ablation. The table is based on the `s0` TOML file for each family; the corresponding `s1` and `s2` files keep the same configuration except for the training seed and run name.

Evidence. Figure 4.5 compares the `best_00` baseline against each heavy-GINE alternative. Each panel shows the seed-averaged test-survival curve for `best_00` and one comparison family, with faint seed curves and variability bands.

Interpretation. The broad search suggests that the heavy baseline is not limited by capacity alone. Several simpler or more constrained variants improve final survival over `best_00`, especially removing node-ID embeddings, removing flat concatenation, using an MLP critic, and combining a light GINE actor with an MLP critic. In contrast, simply switching message passing to GAT or weighted GCN, or removing the initial do-nothing bias while keeping the heavy GINE/GNN-critic structure, does not clearly solve the stability problem.

Table 4.7: Heavy-GINE best-search 2 parameter comparison from `configs/gine_s0_s1_s2`.

Run	Main change from <code>best_00</code>	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
<code>best_000</code>	Removes flat-observation concatenation	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	no	yes	yes	yes	0.02–0.02	0.7	no
<code>best_00</code>	Reference: heavy concat-flat GINE with GNN critic	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
<code>best_01</code>	Enables optimized critic updates	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	yes
<code>best_02</code>	Replaces the GNN critic with an MLP critic	<code>gine</code>	2×128	<code>mlp</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
<code>best_03</code>	Uses non-shared actor GNN weights	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	no	yes	yes	0.02–0.02	0.7	no
<code>best_04</code>	Uses a lighter GINE encoder	<code>gine</code>	1×64	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
<code>best_05</code>	Decays entropy from 0.02 to 0.0	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.00	0.7	no
<code>best_06</code>	Removes learned node-ID embeddings	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	no	0.02–0.02	0.7	no
<code>best_07</code>	Replaces GINE with GAT message passing	<code>gat</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
<code>best_08</code>	Replaces GINE with weighted GCN message passing	<code>gcn</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
<code>best_09</code>	Light no-concat GINE with MLP critic and optimized critic updates	<code>gine</code>	1×64	<code>mlp</code>	$2 \times 128 / 2 \times 128$	no	yes	no	yes	0.02–0.02	0.7	yes
<code>best_10</code>	Light concat-flat GINE with MLP critic and optimized critic updates	<code>gine</code>	1×64	<code>mlp</code>	$2 \times 128 / 2 \times 128$	yes	yes	no	yes	0.02–0.02	0.7	yes
<code>best_11</code>	Removes the initial do-nothing bias	<code>gine</code>	2×128	<code>gnn</code>	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.0	no

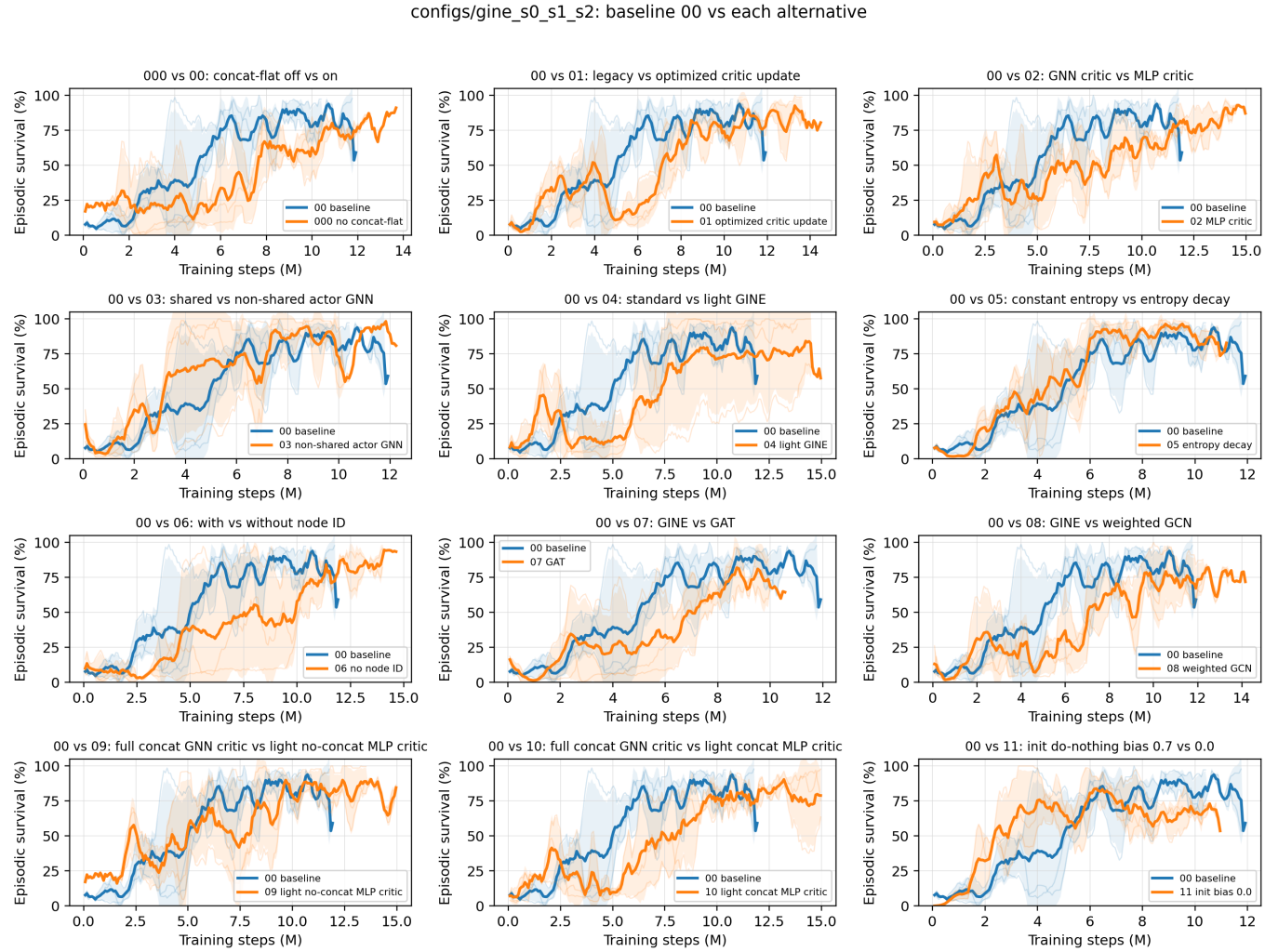


Figure 4.5: Heavy-GINE best-search 2 survival comparisons. Each subplot compares the **best_00** heavy concat-flat GINE baseline against one controlled alternative from configs/gine_s0_s1_s2.

4.5 WCCI Action-Space Reduction

The WCCI bus36 setting has a much larger discrete topology-action space than bus14. Before training MAPPO on this environment, the action space is reduced offline from simulated action outcomes. During rollouts, states are collected when the grid is stressed,

$$\rho_{\max} > 0.90.$$

For each collected state s , candidate actions are simulated and compared against do nothing:

$$\Delta(s, a) = \rho_{\text{after}}(s, a) - \rho_{\text{after}}(s, 0).$$

An action is counted as useful when it lowers the post-action loading relative to do nothing, i.e. $\Delta(s, a) < -\epsilon$. Each action is then scored by its empirical improvement rate,

$$\text{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

with a minimum-count filter to avoid selecting rarely sampled actions. The final reduced action space keeps the best actions per agent, while always preserving action 0. The candidate sets are then sanity-checked with a one-step simulator-greedy controller: at each risky state, the controller simulates the available reduced actions and executes the best improving one.

Table 4.8: One-step greedy evaluation of WCCI reduced action spaces on 50 random test chronics. All rows use the same chronic sample (`chronic_sample_seed=0`); the do-nothing mean survival on this sample is 28.30%.

Reduced action space	Kept actions per agent (a_0, a_1, a_2, a_3)	Greedy survival (%)	Do-nothing survival (%)	Survival delta (%)	Greedy full survival (%)
Do-nothing reference	1, 1, 1, 1	—	28.30	—	—
WCCI $h = 1$, mk64	64, 64, 64, 64	55.24	28.30	26.94	22.00
WCCI $h = 1$, mk128	77, 128, 127, 128	60.65	28.30	32.35	22.00
WCCI $h = 1$, mk256	77, 256, 127, 256	68.61	28.30	40.31	36.00
WCCI $h = 1$, mk512	77, 512, 127, 512	71.21	28.30	42.90	40.00
WCCI $h = 1$, mk1024	77, 1024, 127, 1024	78.18	28.30	49.87	54.00

4.6 Decentralized Sparse Intervention Control

Question. Can the agents keep high survival while behaving more like operators: acting rarely, acting locally, acting mainly when the grid state makes an intervention useful, and keeping the final policy decentralized?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i , the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \quad a_i \in \{0, 1, \dots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{\text{eval}} = \arg \max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests four ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, and adaptive Lagrangian intervention budgets.

Motivation. The intervention gate is motivated by hierarchical topology-control work, where the action can be decomposed into “do nothing versus intervene”, then where to act, then which local topology configuration to apply [3], [4]. In this thesis only the first level is adapted: each regional agent independently decides whether to do nothing or to execute one of its existing local topology actions. The fixed and adaptive sparse-control objectives are closer to constrained and budgeted RL: non-idle topology interventions are treated as a limited resource, following the general Lagrangian view of constrained policy optimization and budgeted control [5], [6], [7]. This is also operationally meaningful because switching frequency and topology deviation are real power-grid objectives, not just cosmetic metrics [8].

4.6.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise,} \end{cases} \quad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

4.6.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i | o_i), \quad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i | o_i), \quad b_i \in \{1, \dots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i | g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \quad P(a_i = a > 0) = P(g_i = 1)P(b_i = a | g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1)H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate}H(g_i) + \alpha_{nonidle}H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

4.6.3 Fixed Sparse-Action Penalty

Implementation detail. The Sparse16 sweep adds a fixed reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed} \mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \quad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \quad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. All Sparse16 runs set `safe_intervention_penalty = 0.0`, so the penalty is not rho-weighted.

Interpretation. Sparse16 changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. The key comparison is `sparse16_flat_p010` versus `sparse16_gated_p010`, which asks whether the gate adds value once a simple sparse objective already exists.

4.6.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent,

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0] w_i(s_t),$$

and the desired constraint is

$$\mathbb{E}_{\pi_i} [c_i(s_t, a_{i,t})] \leq d.$$

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \text{clip} \left(\lambda_i + \eta (\hat{C}_i - d), 0, \lambda_{\max} \right), \quad \hat{C}_i = \frac{1}{T} \sum_{t=1}^T c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d , future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Three cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Global-safe cost:** $w_i^{global}(s_t) = \sigma(k(\rho_{safe} - \rho_t^{global}))$. This penalizes non-idle actions mostly when the whole grid is safe.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The first AIB folder, `configs/adaptive_intervention_budget_7`, contains the main local-budget ablations:

- `aib_00_flat_local_t020`: flat actor, local-safe cost, $d = 0.20$;
- `aib_01_flat_local_t010`: stricter local-safe budget, $d = 0.10$;
- `aib_02_flat_local_t035`: looser local-safe budget, $d = 0.35$;
- `aib_03_gate_hgreedy_sep_local_t020`: gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- `aib_04_flat_nonidle_t020`: flat actor with adaptive plain non-idle cost, $d = 0.20$.

The comparison `aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020` isolates the value of local rho weighting.

The mechanism-ablation folder `configs/adaptive_intervention_budget_mechanism_15` contains 15 additional flat-actor runs. It separates fixed versus adaptive coefficients, plain non-idle versus state-dependent costs, global versus local rho weighting, and rho-threshold sensitivity.

Table 4.9: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Sparse16	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design

Table 4.10: Main sparse-control comparisons to report.

Question	Comparison
Does a fixed sparse penalty already work?	Baseline versus <code>sparse16_flat_p010</code>
Does the gate help beyond a sparse objective?	<code>sparse16_flat_p010</code> versus <code>sparse16_gated_p010</code>
Fixed versus adaptive coefficient?	<code>sparse16_flat_p010</code> or <code>aibm_00_fixed_nonidle_p006</code> versus <code>aib_04_flat_nonidle_t020</code>
Does local-safe weighting matter?	<code>aib_00_flat_local_t020</code> versus <code>aib_04_flat_nonidle_t020</code>
Does global-safe weighting matter?	<code>aibm_02_adaptive_global_t020</code> versus <code>aib_00_flat_local_t020</code>
Is $\rho_{safe} = 0.90$ special?	<code>aibm_03</code> with 0.85, <code>aib_00</code> with 0.90, and <code>aibm_04</code> with 0.95
Is the heuristic only an evaluation trick?	Rho-heuristic runs versus learned sparse/AIB runs

Full-test checkpoint summaries. The following tables report the average full-test episodic survival over the three evaluated seeds. The survival values are computed directly from the JSON files in `Topology_Task/outputs/full_test_eval`. These JSON files also record where the compact action-distribution artifacts were written. The action-0 fractions are computed from the corresponding local `Topology_Task/outputs/full_test_eval_actions/*/action_summary.json` files and averaged over

seeds. The all-agent action-0 column is the arithmetic mean of the three per-agent action-0 fractions. The do-nothing row is a reference policy that always selects action 0; its survival is computed from `Topology_Task/outputs/do_nothing_eval/bus14_test_20`. Remaining “–” entries indicate that the compact action summaries for those full-test jobs are not available locally.

Table 4.11: Full-test summary for evaluation-time rho heuristics.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Global rho heuristic, $\rho_{safe} = 0.90$	98.36	0.936	0.933	0.936	0.940
Local rho heuristic, $\rho_{safe} = 0.90$	97.07	0.968	0.992	0.985	0.928

Table 4.12: Full-test summary for intervention-gated actors.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Intervention gate, final-action MAP	89.82	0.802	0.826	0.889	0.690
Intervention gate, hierarchical greedy	93.66	0.395	0.066	0.844	0.276

Table 4.13: Full-test summary for Sparse16 flat-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat Sparse16, $\lambda_{fixed} = 0.000$	98.23	0.494	0.524	0.463	0.496
Flat Sparse16, $\lambda_{fixed} = 0.001$	96.75	0.486	0.642	0.401	0.416
Flat Sparse16, $\lambda_{fixed} = 0.003$	93.97	0.575	0.691	0.346	0.688
Flat Sparse16, $\lambda_{fixed} = 0.010$	97.44	0.905	0.907	0.954	0.853

Metrics to report. The main performance metric remains episodic survival. It should be paired with sparsity and physical-diagnostic metrics: per-agent action-0 fractions, per-agent non-idle rates, the distribution of how many agents act simultaneously, intervention-budget multipliers, budget costs and violations, the rate of interventions in costly safe states, pre- and post-action max rho, and topology-distance changes.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, `sparse16_flat_p010` is a strong fixed-penalty baseline, while the most principled learned sparse-control candidate is `aib_00_flat_local_t020`: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight.

4.7 Current Takeaway

The current reruns indicate that the strongest direction is not a single isolated modification. Stable performance comes from a combination of A0-style training choices, reward normalization, careful initialization, and architectures that keep the critic and graph encoder simple enough to optimize reliably. Optimized critic updates and lighter GINE/MLP-critic variants can improve the final GINE rerun performance, but the A0 baseline remains the clearest stable reference point for future ablations. The sparse-control roadmap adds a second axis to the thesis: after survival is stabilized, the next question is whether the same decentralized agents can achieve high survival with fewer unnecessary topology interventions.

Table 4.14: Full-test summary for Sparse16 gated-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Gated Sparse16, $\lambda_{fixed} = 0.000$	78.51	0.787	0.747	0.949	0.665
Gated Sparse16, $\lambda_{fixed} = 0.001$	73.82	0.862	0.956	0.832	0.799
Gated Sparse16, $\lambda_{fixed} = 0.003$	94.59	0.887	0.934	0.929	0.799
Gated Sparse16, $\lambda_{fixed} = 0.010$	85.69	0.974	0.990	0.984	0.947

Table 4.15: Full-test summary for adaptive intervention budget runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat local-safe AIB, $d = 0.20$	98.31	0.978	0.980	0.989	0.966
Flat local-safe AIB, $d = 0.10$	98.39	0.979	0.992	0.995	0.949
Flat local-safe AIB, $d = 0.35$	96.61	0.934	0.959	0.947	0.896
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	33.87	0.999	1.000	1.000	0.998
Flat non-idle AIB, $d = 0.20$	95.08	0.986	0.997	0.992	0.970

Table 4.16: Full-test summary for the teacher-student distillation runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Teacher: MAPPO with local rho heuristic	97.07	0.968	0.992	0.985	0.928
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	92.14	0.939	0.974	0.925	0.919
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	90.76	0.957	0.982	0.948	0.940

Appendix A

GINE and GAT Graph-Actor Architectures

This appendix specifies the complete edge-aware actor used in the graph experiments. GINE and GAT receive the same graph inputs and use the same pooling and action head. Their only architectural difference is the message-passing operator: GINE inserts an edge embedding into the message content, whereas GAT uses an edge embedding to decide how strongly the message is weighted.

A.1 Shared input and encoder pipeline

For agent a , the encoder receives the local graph $\mathcal{G}_t^{(a)}$. Candidate edges with `edge_mask`=0 are removed before message passing, so the layer sees only the active directed set $\mathcal{E}_t^{(a)}$. The source of $u \rightarrow v$ sends a message and v aggregates it. Node and edge masks do not become learned features.

Each raw node vector $\mathbf{x}_v$ first passes through

$$\mathbf{h}_v^{(0)} = \text{LayerNorm}(\text{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \quad (\text{A.1})$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE or GAT layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \text{LayerNorm}(\text{ReLU}(\tilde{\mathbf{h}}_v^{(k+1)})). \quad (\text{A.2})$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

A.2 GINE message passing

For an active edge $u \rightarrow v$, layer k projects the complete edge feature vector and adds it to the sender state:

$$\tilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \quad (\text{A.3})$$

$$\mathbf{m}_{u \rightarrow v}^{(k)} = \text{ReLU}(\mathbf{h}_u^{(k)} + \tilde{\mathbf{e}}_{uv}^{(k)}), \quad (\text{A.4})$$

$$\tilde{\mathbf{h}}_v^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon) \mathbf{h}_v^{(k)} + \sum_{u: (u,v) \in \mathcal{E}_t} \mathbf{m}_{u \rightarrow v}^{(k)} \right). \quad (\text{A.5})$$

The implementation uses the PyTorch Geometric default $\epsilon = 0$ without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation A.5 shows why edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise,

reversing a typed heterogeneous edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

A.3 GAT message passing

GAT first computes a score for every active incoming edge and attention head. For head r , the implementation has the form

$$s_{uv}^{(k,r)} = \text{LeakyReLU}\left(\mathbf{a}_{s,r}^\top \mathbf{W}_{s,r} \mathbf{h}_u^{(k)} + \mathbf{a}_{d,r}^\top \mathbf{W}_{d,r} \mathbf{h}_v^{(k)} + \mathbf{a}_{e,r}^\top \mathbf{W}_{e,r} \mathbf{e}_{uv}\right), \quad (\text{A.6})$$

$$\alpha_{uv}^{(k,r)} = \frac{\exp s_{uv}^{(k,r)}}{\sum_{w:(w,v) \in \mathcal{E}_t} \exp s_{vw}^{(k,r)}}, \quad (\text{A.7})$$

$$\tilde{\mathbf{h}}_v^{(k+1)} = \frac{1}{H} \sum_{r=1}^H \sum_{u:(u,v) \in \mathcal{E}_t} \alpha_{uv}^{(k,r)} \mathbf{W}_r \mathbf{h}_u^{(k)}. \quad (\text{A.8})$$

GAT uses $H = 4$ heads and `concat=false`, so the four head outputs are averaged and the layer still returns 128 features. The edge embedding affects α_{uv} : it changes how much attention the receiver assigns to the sender. In the standard GAT operator used here, the edge vector is not added to the value vector $\mathbf{W}_r \mathbf{h}_u$. This is the central difference from GINE.

GAT internally adds a self-loop for every node. That computational self-loop is separate from the optional explicit `relation_self` edge in the graph schema.

Table A.1: Exact architectural difference between the implemented GINE and GAT actors.

Component	GINE	GAT
Edge use	Added to the sender state inside every message	Enters the attention score for every incoming edge
Neighbor aggregation	Unweighted sum	Softmax-weighted sum
Self information	Explicit $(1 + \epsilon) \mathbf{h}_v$ term	Internal GAT self-loop
Multiple heads	Not used	Four heads, averaged rather than concatenated
Post-layer operation	ReLU, then LayerNorm	ReLU, then LayerNorm
Pooling and action head	Shared implementation	Shared implementation

A.4 Pooling and graph output

With mean pooling, the graph representation after the second message-passing layer is

$$\bar{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \quad (\text{A.9})$$

where m_v is the valid-node mask. This is one mean over all valid node types. It is not a separate mean per type. Therefore:

- the busbar graph pools all busbars;
- the direct heterogeneous graph pools busbars, generators, and loads;
- the explicit-line graph also pools transmission-line nodes.

For a local actor graph, one-hop contextual nodes are included in the mean. The ordinary GINE/GAT mean readout does not restrict pooling to controlled nodes. Sum and maximum pooling change only Equation A.9; they do not change message passing.

The pooled 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \text{ReLU}(\mathbf{W}_o \bar{\mathbf{h}}_G + \mathbf{b}_o), \quad \mathbf{z}_a \in \mathbb{R}^{128}. \quad (\text{A.10})$$

When virtual-node readout is selected, the final virtual-node state replaces $\bar{\mathbf{h}}_G$; the output projection and its dimension stay the same.

A.5 Agent-specific action head

The flat observation is not concatenated to $\mathbf{z}_a$. Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \text{ReLU}(\mathbf{W}_a^{(1)}\mathbf{z}_a + \mathbf{b}_a^{(1)}), \quad (\text{A.11})$$

$$\mathbf{q}_a^{(2)} = \text{ReLU}(\mathbf{W}_a^{(2)}\mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \quad (\text{A.12})$$

$$\ell_a = \mathbf{W}_a^{(3)}\mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \quad \ell_a \in \mathbb{R}^{|\mathcal{A}_a|}, \quad (\text{A.13})$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \text{Categorical}(\text{logits} = \ell_a). \quad (\text{A.14})$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects $\arg \max_j \ell_{a,j}$. Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths $|\mathcal{A}_a|$. The critic is a separate MLP over the centralized flat state and does not reuse the actor GINE/GAT embedding.

Equation A.14 is the default head. The optional candidate-action head of Section 2.5 keeps this pipeline up to $\mathbf{z}_a$ and additionally consumes the post-message-passing node states $\mathbf{h}_v^{(K)}$, which are the same states that Equation A.9 averages.

A.6 Exact node inputs by representation

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE and GAT through the `is_*` columns.

Table A.2: Node types and nonzero input features received by GINE and GAT.

Representation	Node type	Populated input features
<code>bus</code>	Busbar	<code>gen_p</code> , <code>gen_theta</code> , <code>load_p</code> , <code>load_theta</code> , <code>time_before_cooldown_sub</code> , <code>domain_mask</code>
<code>heterogeneous</code>	Busbar	<code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>connected</code> , <code>domain_mask</code>
	Generator	<code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Load	<code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>connected</code> , <code>domain_mask</code>
<code>heterogeneous_line</code>	Busbar	<code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>connected</code> , <code>domain_mask</code>
	Generator	<code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Load	<code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Transmission line	<code>is_transmission_line</code> , <code>line_status</code> , <code>rho</code> , <code>timestep_overflow</code> , <code>time_before_cooldown_line</code> , <code>optional maintenance times</code> , <code>connected</code> , <code>domain_mask</code>
Any augmented schema	Substation summary	Learned substation-indexed vector; no raw physical feature row
	Virtual node	One learned vector shared across graphs; no raw physical feature row

A.7 Exact edge inputs by representation

The edge vector contains all edge information consumed by GINE and GAT. Inactive candidates are filtered out before the vectors in Table A.3 reach a convolution.

Table A.3: Directed edge relations and edge-feature vectors received by GINE and GAT. “Relation” denotes one active relation-one-hot column.

Representation	Directed relation	Edge features
bus	Physical busbar $\leftrightarrow$ busbar	<code>line_status</code> , <code>rho</code> , <code>timestep_overflow</code> , <code>time_before_cooldown_line</code> , optional maintenance times; no relation one-hot
	Optional self or same-substation	Same width; physical values zero and <code>rho</code> = 1; no relation one-hot
heterogeneous	Physical busbar $\leftrightarrow$ busbar	Complete measured physical-line block plus <code>relation_physical_line</code>
	Generator $\leftrightarrow$ busbar	Neutral physical block plus direction-specific generator relation
	Load $\leftrightarrow$ busbar	Neutral physical block plus direction-specific load relation
	Optional self or same-substation	Neutral physical block plus the corresponding relation
heterogeneous_line	Busbar $\leftrightarrow$ line node	One origin/extremity-and-direction relation; no line-state attributes
	Generator $\leftrightarrow$ busbar	One direction-specific generator relation
	Load $\leftrightarrow$ busbar	One direction-specific load relation
	Optional self or same-substation	One corresponding relation
Any augmented schema	Busbar $\rightarrow$ summary/virtual, with optional reverse	Zero edge vector except neutral <code>rho</code> = 1 when present; no dedicated relation one-hot in the ordinary GINE/GAT encoder

The complete relation names and activation rules are given in Tables 2.5 and 2.7. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to both edge-aware models: GINE changes message content and GAT changes attention.

Appendix B

Full-Test Checkpoint Registry

Table [B.1](#) reports the checkpoint step used for each full-test result table. The values are the `checkpoint_global_step` fields stored in the corresponding JSON files under `Topology_Task/outputs/full_test_eval`. The do-nothing policy has no learned checkpoint. For the behavioral cloning students, the stored step is the student checkpoint’s optimizer-step count rather than a MAPPO environment step.

Table B.1: Checkpoint steps used for the full-test evaluations.

Run type	Seed 0 checkpoint step	Seed 1 checkpoint step	Seed 2 checkpoint step
Do-nothing policy	—	—	—
Baseline flat policy	13,600,000	13,040,000	13,760,000
Global rho heuristic, $\rho_{safe} = 0.90$	13,920,000	14,320,000	14,800,000
Local rho heuristic, $\rho_{safe} = 0.90$	14,160,000	14,400,000	14,160,000
Intervention gate, final-action MAP	12,000,000	12,720,000	13,760,000
Intervention gate, hierarchical greedy	7,120,000	14,720,000	13,760,000
Flat Sparse16, $\lambda_{fixed} = 0.000$	9,600,000	12,160,000	7,920,000
Flat Sparse16, $\lambda_{fixed} = 0.001$	10,640,000	11,520,000	11,600,000
Flat Sparse16, $\lambda_{fixed} = 0.003$	12,000,000	12,960,000	12,480,000
Flat Sparse16, $\lambda_{fixed} = 0.010$	8,960,000	5,760,000	12,560,000
Gated Sparse16, $\lambda_{fixed} = 0.000$	8,480,000	4,320,000	6,000,000
Gated Sparse16, $\lambda_{fixed} = 0.001$	5,760,000	7,600,000	3,280,000
Gated Sparse16, $\lambda_{fixed} = 0.003$	8,960,000	7,120,000	7,600,000
Gated Sparse16, $\lambda_{fixed} = 0.010$	12,000,000	7,840,000	7,920,000
Flat local-safe AIB, $d = 0.20$	13,120,000	9,360,000	12,160,000
Flat local-safe AIB, $d = 0.10$	12,880,000	13,200,000	12,720,000
Flat local-safe AIB, $d = 0.35$	12,800,000	5,360,000	11,520,000
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	14,960,000	5,280,000	16,800,000
Flat non-idle AIB, $d = 0.20$	12,480,000	14,880,000	11,920,000
Teacher: MAPPO with local rho heuristic	14,160,000	14,400,000	14,160,000
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	12,375	14,994	13,617
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	12,375	14,994	13,617

Bibliography

- [1] B. Donnot, “Grid2op: A testbed platform to model sequential decision making in power systems,” *arXiv preprint arXiv:2011.07929*, 2020.
- [2] C. Yu et al., “The surprising effectiveness of ppo in cooperative multi-agent games,” *Advances in Neural Information Processing Systems*, 2022.
- [3] J. Manczak et al., *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: [2311.02129](https://arxiv.org/abs/2311.02129). [Online]. Available: <https://arxiv.org/abs/2311.02129>
- [4] E. van der Sar et al., *Hierarchical multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: [2310.02605](https://arxiv.org/abs/2310.02605). [Online]. Available: <https://arxiv.org/abs/2310.02605>
- [5] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” *International Conference on Machine Learning*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [6] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID lagrangian methods,” *International Conference on Machine Learning*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.03964>
- [7] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, “Budgeted reinforcement learning in continuous state space,” *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01004>
- [8] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: [2502.00040](https://arxiv.org/abs/2502.00040). [Online]. Available: <https://arxiv.org/abs/2502.00040>
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [10] A. Marot et al., “Learning to run a power network challenge for training topology controllers,” *Electric Power Systems Research*, 2021.
- [11] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *International Conference on Learning Representations*, 2017.